```
# 文件 app.py
# -*- coding: utf-8 -*-
"""Incremental defect detection: model import, tasks, evaluation and daily use."""
import hashlib
import json
import os
import sys
import threading
from pathlib import Path
os.environ.setdefault("MPLBACKEND", "Agg")
# Full supplied checkpoints restore every backbone tensor; skip redundant downloads.
os.environ.setdefault("STEEL_PRETRAINED_BACKBONE", "0")
os.environ.setdefault("GRADIO_ANALYTICS_ENABLED", "False")
import gradio as gr
import numpy as np
import torch
from PIL import Image, ImageDraw, ImageFont
from adapters.deformable_detr import CLASSES, SPLITS
from services.workflow_store import WorkflowStore
PROJECT = Path(__file__).resolve().parent
sys.path.insert(0, str(PROJECT))
sys.path.insert(0, str(PROJECT / "models" / "ops"))
BASE_CKPT = PROJECT / "MODEL" / "base_output_group1" / "checkpoint_phase0.pth"
NEU_DET = PROJECT / "NEU-DET"
APP_RUNS = PROJECT / "app_runs"
CLS_COLORS = ["#e6194b", "#f58231", "#ffe119", "#3cb44b", "#4363d8", "#911eb4"]
_model_cache, _model_cache_versions = {}, {}
COMPUTE_LOCK = threading.RLock()
def workflow_store():
    return WorkflowStore(PROJECT / "app_runs" / "workflow")
def ensure_compute_available():
    from services.task_service import active_task
    task = active_task(workflow_store())
    if task:
        raise gr.Error(f"任务「{task['name']}」正在{task['stage']}。训练期间请在监控页查看进度
    ，完成后再检测或导入模型。")
def clear_model_cache():
    _model_cache.clear()
    _model_cache_versions.clear()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
def inference_preferences():
    path = PROJECT / "configs" / "app_inference.json"
    return json.loads(path.read_text(encoding="utf-8")) if path.exists() else {}
def checkpoint_path(value):
    path = Path(value)
    return path if path.is_absolute() else PROJECT / path
def default_checkpoint(checkpoints):
    store = workflow_store()
    active = store.setting("active_version_id")
```

```
    if active:
        version = store.version(active)
        for candidate in checkpoints:
            if checkpoint_path(candidate).resolve() == Path(version["checkpoint"]).resolve():
                return candidate
    preferred = inference_preferences().get("checkpoint")
    if preferred:
        for candidate in checkpoints:
            if checkpoint_path(candidate).resolve() == checkpoint_path(preferred).resolve():
                return candidate
    dual = [c for c in checkpoints if "dual" in c.lower() and Path(c).name == "best_overall_pha
    se1.pth"]
    if dual:
        return max(dual, key=lambda c: checkpoint_path(c).stat().st_mtime_ns)
    return next((c for c in checkpoints if "v111_best_new" in c), checkpoints[0] if checkpoints
     else None)
def checkpoint_description(value):
    if not value:
        return "尚未找到模型，可在【项目与基座】导入权重并检查兼容性。"
    for version in workflow_store().versions():
        if checkpoint_path(value).resolve() == Path(version["checkpoint"]).resolve():
            return (f"**模型：{version['name']}** · `{version['id']}`\n\n"
                    f"支持类别：{' / '.join(version['class_names'])}\n\n{version['limitation']}
    ")
    preferences = inference_preferences()
    preferred = preferences.get("checkpoint")
    if preferred and checkpoint_path(value).resolve() == checkpoint_path(preferred).resolve():
        epoch = preferences.get("epoch")
        return (f"**当前模型：{preferences.get('name', '已验证模型')}**"
                + (f" · 第 {epoch + 1} 轮最佳权重" if epoch is not None else "")
                + "\n\n 支持六类缺陷。可在【项目与基座】登记为可追溯的模型版本。")
    return f"当前权重：`{value}`；类别信息加载后由存档配置恢复。"
def checkpoint_choices():
    names = {str(Path(v["checkpoint"]).resolve()): f"{v['name']} · {len(v['class_order'])} 类 ·
     {v['id']}"
             for v in workflow_store().versions()}
    return [(names.get(str(checkpoint_path(path).resolve()), path), path) for path in list_chec
    kpoints()]
def refresh_checkpoints(current):
    choices = list_checkpoints()
    selected = current if current in choices else default_checkpoint(choices)
    return gr.update(choices=checkpoint_choices(), value=selected), checkpoint_description(sele
    cted)
def get_model(ckpt_path):
    from adapters.deformable_detr import load_model
    path = checkpoint_path(ckpt_path).resolve()
    stat = path.stat()
    key, signature = str(path), (stat.st_mtime_ns, stat.st_size)
    if key in _model_cache and _model_cache_versions.get(key) == signature:
        return _model_cache[key]
```

```
    preferences = inference_preferences()
    preferred = preferences.get("checkpoint")
    version = next((v for v in workflow_store().versions() if Path(v["checkpoint"]).resolve() =
    = path), None)
    hashes = [version["sha256"]] if version else []
    if preferred and path == checkpoint_path(preferred).resolve() and preferences.get("sha256")
    :
        hashes.append(preferences["sha256"])
    if hashes:
        with path.open("rb") as stream:
            actual = hashlib.file_digest(stream, "sha256").hexdigest()
        if any(actual != expected for expected in hashes):
            raise gr.Error("模型权重与已登记或已验证版本不一致，请检查文件并重新导入。")
    model, args, _ = load_model(path, class_names=",".join(version["class_names"]) if version e
    lse "",
                                config=version["model_config"] if version else None)
    clear_model_cache()
    _model_cache[key] = (model, args)
    _model_cache_versions[key] = signature
    return model, args
def list_checkpoints():
    paths = set()
    for pattern in ("checkpoints/*.pth", "MODEL/full_train_output/*.pth", "MODEL/daily_dual/*.p
    th", "incremental_output_*/*.pth",
                    "incremental_output_dual_pilot_*/*/best_overall_phase1.pth", "app_runs/*/ou
    tput/*.pth",
                    "app_runs/workflow/tasks/*/output/best_overall_phase1.pth", "MODEL/base_out
    put_group*/checkpoint_phase0.pth"):
        paths.update(p.relative_to(PROJECT).as_posix() for p in PROJECT.glob(pattern))
    for version in workflow_store().versions():
        path = Path(version["checkpoint"])
        if path.exists():
            paths.add(path.relative_to(PROJECT).as_posix() if path.is_relative_to(PROJECT) else
     str(path))
    return sorted(paths)
def draw_detections(img_np, boxes, labels, scores):
    pil = Image.fromarray(img_np.copy())
    scale = max(1, 420 // max(pil.size))
    if scale > 1:
        pil = pil.resize((pil.width * scale, pil.height * scale), Image.Resampling.BILINEAR)
    draw = ImageDraw.Draw(pil)
    try:
        font = ImageFont.truetype("arial.ttf", 11 * scale)
    except OSError:
        font = ImageFont.load_default()
    for (x1, y1, x2, y2), label, score in zip(boxes, labels, scores):
        color = CLS_COLORS[label % len(CLS_COLORS)]
        draw.rectangle([x1*scale, y1*scale, x2*scale, y2*scale], outline=color, width=2)
        draw.text((x1*scale+1, max(0, y1*scale-13*scale)), f"{CLASSES[label]} {score:.2f}", fil
    l=color, font=font)
```

```
    return np.array(pil)
def run_inference(ckpt_path, image, confidence_threshold=.2, max_detections=20):
    from datasets.neu_det import make_neu_det_transforms
    from models.deformable_detr import PostProcess
    from util.misc import nested_tensor_from_tensor_list
    if not ckpt_path:
        return None, [["请先选择模型", "", "", "", "", ""]]
    if image is None:
        return None, [["请先上传图片", "", "", "", "", ""]]
    with COMPUTE_LOCK:
        ensure_compute_available()
        model, args = get_model(ckpt_path)
        pil = Image.open(image).convert("RGB") if isinstance(image, (str, Path)) else Image.fro
    marray(image).convert("RGB")
        image_tensor, _ = make_neu_det_transforms("val", val_size=224)(pil, None)
        samples = nested_tensor_from_tensor_list([image_tensor]).to(args.device)
        with torch.no_grad():
            outputs = model(samples)
        result = PostProcess(nms_mode="perclass", max_detections=100)(
            outputs, torch.tensor([[pil.height, pil.width]], device=args.device))[0]
        boxes, local_labels, scores = (result[k].detach().cpu() for k in ("boxes", "labels", "s
    cores"))
        mapping = getattr(args, "class_global_ids", list(range(6)))
        labels = torch.tensor([mapping[i] for i in local_labels.tolist()], dtype=torch.long)
        valid = scores >= float(confidence_threshold)
        boxes, labels, scores = boxes[valid], labels[valid], scores[valid]
        if not len(scores):
            return np.array(pil), [["（当前阈值下无检测）", "", "", "", "", ""]]
        order = torch.argsort(scores, descending=True)[:max(1, int(max_detections))]
        boxes, labels, scores = boxes[order].numpy(), labels[order].tolist(), scores[order].tol
    ist()
        rows = [[CLASSES[label], f"{score:.3f}", *[f"{v:.1f}" for v in box]] for box, label, sc
    ore in zip(boxes, labels, scores)]
        return draw_detections(np.array(pil), boxes, labels, scores), rows
def build_command(ds_root, run_dir, split_group, epochs, samples_cap, adv):
    """Legacy command API kept for existing scripts; new UI uses immutable tasks."""
    cmd = [sys.executable, "-u", "incremental_train.py", "--incremental_only", "--split_group",
     str(split_group),
           "--new_class_samples", str(samples_cap), "--epochs_incremental", str(epochs),
           "--output_dir", str(run_dir / "output"), "--resume",
           str(PROJECT / "MODEL" / f"base_output_group{split_group}" / "checkpoint_phase0.pth")
    ,
           "--neu_det_path", str(ds_root), "--post_nms_mode", "perclass", "--accum_steps_inc",
    "2", "--shape_gate", "1"]
    if adv.get("incremental_arch", "baseline") == "baseline":
        cmd.append("--decoder_tune_inc")
    effective = {"lr_incremental": "4e-5", "cls_lr_scale": "200", "new_class_prior": ".01",
                 "bbox_lr_scale": "1", "bbox_pullback": "1e-3", "distill_weight": ".5",
                 "feat_distill_weight": ".3", "giou_loss_coef": "4", "sharpen_loss_coef": "3",
                 "shape_lr_scale": "20", "shape_loss_coef": "2", "aspect_loss_coef": "4", **adv
```

```
    }
    if effective.get("incremental_arch") == "dual":
        effective.update(dual_lr="1e-4", dual_head_lr="5e-4", warmstart_epochs="0", cls_loss_co
    ef="2",
                         bbox_loss_coef="5", giou_loss_coef="2", set_cost_class="2", set_cost_b
    box="5", set_cost_giou="2")
    for key, value in effective.items():
        cmd.extend([f"--{key}", str(value)])
    return cmd
def build_ui():
    from ui.workflow import build_workflow_ui
    return build_workflow_ui(sys.modules[__name__])
if __name__ == "__main__":
    build_ui().queue().launch(server_name="127.0.0.1", server_port=7860, inbrowser=True)
# 文件 launch.py
"""Launch the packaged application using the active Python environment."""
import argparse
import os
import socket
import sys
from pathlib import Path
ROOT = Path(__file__).resolve().parent
os.chdir(ROOT)
sys.path.insert(0, str(ROOT))
os.environ.setdefault("PYTHONDONTWRITEBYTECODE", "1")
os.environ.setdefault("PYTHONIOENCODING", "utf-8")
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--port", type=int, default=0)
    parser.add_argument("--no-browser", action="store_true")
    args = parser.parse_args()
    port = args.port
    if not port:
        for candidate in range(7860, 7881):
            with socket.socket() as probe:
                try:
                    probe.bind(("127.0.0.1", candidate))
                    port = candidate
                    break
                except OSError:
                    continue
    if not port:
        parser.error("No available local port in 7860..7880; choose --port.")
    import app
    app.build_ui().queue().launch(server_name="127.0.0.1", server_port=port, inbrowser=not args
    .no_browser)
if __name__ == "__main__":
    main()
# 文件 ui/workflow.py
"""User-facing incremental workflow, backed by persisted model and task records."""
```

```
import json
from functools import wraps
from pathlib import Path
import gradio as gr
import pandas as pd
from adapters.deformable_detr import CLASSES, SPLITS, inspect_checkpoint
from services.task_service import (activate_version, create_incremental_task, launch_task,
                                   recover_interrupted, register_model, resolve_parent_model)
STATE_NAMES = {"draft": "草稿", "validating": "校验中", "ready": "待启动", "running": "运行中",
               "evaluating": "复核中", "succeeded": "已完成", "failed": "失败", "stop_requested
    ": "等待安全停止",
               "stopped": "已停止", "interrupted": "已中断"}
def user_errors(function):
    @wraps(function)
    def wrapped(*args, **kwargs):
        try:
            return function(*args, **kwargs)
        except (ValueError, OSError, KeyError, RuntimeError) as exc:
            raise gr.Error(str(exc)) from exc
    return wrapped
def file_path(value):
    return str(value) if isinstance(value, (str, Path)) else str(value.name) if value is not No
    ne else None
def model_choices(store, bases=False):
    return [(f"{v['name']} · {len(v['class_order'])} 类 · {v['id']}", v["id"])
            for v in store.versions() if not bases or v["capabilities"].get("incremental_base")
    ]
def task_choices(store):
    return [(f"{t['name']} · {STATE_NAMES.get(t['state'], t['state'])} · {t['id']}", t["id"]) f
    or t in store.tasks()]
def parent_choices(store, checkpoint):
    choices = model_choices(store, True)
    path = Path(checkpoint).resolve()
    registered = {Path(item["checkpoint"]).resolve() for item in store.versions()
                  if item["capabilities"].get("incremental_base")}
    if path.is_file() and path not in registered:
        choices.append(("随附旧类基座 · checkpoint_phase0.pth（待检查登记）", str(path)))
    return choices
def parent_description(store, version_id):
    if not version_id:
        return "请选择旧类基座，或粘贴基座 .pth 文件路径。也可以在【项目与基座】上传并登记模型
    。"
    version = next((item for item in store.versions() if item["id"] == version_id), None)
    if version is None:
        return (f"**待检查的基座**：`{version_id}`\n\n 点击【检查并接入父模型】，或直接校验训练
    方案："
                "系统会严格加载、试推理并登记兼容基座，成功后才能创建训练任务。"
                "\n\n 需要补充类别顺序或结构配置时，请到【项目与基座】导入。")
    if not version["capabilities"].get("incremental_base"):
        return "该版本仅可用于检测，请选择旧三类 phase-0 基座。"
```

```
    split = SPLITS[version["split_group"]]
    return (f"**父模型：{version['name']}**\n\n 旧类：{' / '.join(version['class_names'])}"
            f"\n\n 本次新增：{' / '.join(CLASSES[i] for i in split['new'])}"
            "\n\n 共享特征和旧分支冻结，训练新增分支；当前支持一次类别增量。")
def inspection_markdown(info):
    capabilities = info["capabilities"]
    return (f"**严格加载和试推理已通过**\n\n 结构：`{info['architecture']}`；类别顺序："
            f"{' / '.join(info['class_names'])}\n\n 来源：{info['class_source']}"
            f"\n\n 可检测：{'是' if capabilities['inference'] else '否'}；可作为增量基座："
            f"{'是' if capabilities['incremental_base'] else '否'}\n\n{info['limitation']}"
            f"\n\nSHA-256：`{info['sha256']}`")
def preflight_markdown(store, task):
    path = Path(task["task_dir"]) / "preflight.json"
    if not path.exists():
        return task.get("error", "数据报告尚未生成。")
    manifest = json.loads(path.read_text(encoding="utf-8"))
    lines = [f"**任务：{task['name']} · {STATE_NAMES.get(task['state'], task['state'])}**", "",
             "标注映射：" + "；".join(f"{key} → {CLASSES[value]}({value})" for key, value in ma
    nifest["label_mapping"].items()), "",
             "| 类别 | 角色 | 训练图片 | 训练框 | 回放图片 | 验证图片 |", "|---|---|---:|---:|-
    --:|---:|"]
    for index, name in enumerate(CLASSES):
        key = str(index)
        counts = manifest["counts"]
        lines.append(f"| {name} | {'旧类' if index in manifest['old_class_ids'] else '新增'} |
    "
                     f"{counts['new'][key]['images']} | {counts['new'][key]['boxes']} | "
                     f"{counts['replay'][key]['images']} | {counts['validation'][key]['images']
    } |")
    lines += ["", f"去重后的实际图片：新图 {len(manifest['new'])}，回放 {len(manifest['replay']
    )}，验证 {len(manifest['validation'])}。",
              f"固定抽样种子：{manifest['sample_seed']}；每个新类抽取额度：{manifest['samples_p
    er_new_class']}。",
              "整个训练复用此清单，每轮随机增强。多类混合图片按并集去重。"]
    if "config" in task:
        config = task["config"]
        lines += ["", f"训练方案：dual，{config['epochs']} 轮，batch={config['batch_size']}，{c
    onfig['device']}；按整体 COCO AP50 选择最佳轮。",
                  f"清单 SHA-256：`{task['manifest_sha256']}`"]
    for label, key in (("检查失败", "errors"), ("数据处理记录", "warnings")):
        if manifest[key]:
            lines += ["", f"**{label}**", ""] + [f"- {message}" for message in manifest[key][:3
    0]]
            if len(manifest[key]) > 30:
                lines.append(f"另有 {len(manifest[key])-30} 项，完整记录见下载报告。")
    return "\n".join(lines)
def monitor_data(store, task_id):
    empty = pd.DataFrame(columns=["epoch", "AP50", "类别组"])
    if not task_id:
        return "请选择任务。", empty, "", []
```

```
    recover_interrupted(store)
    task = store.task(task_id)
    lines = [f"**{task['name']} · {STATE_NAMES.get(task['state'], task['state'])}**", "",
             f"当前阶段：{task['stage']}；父版本：`{task['parent_version_id']}`。"]
    if task.get("epochs"):
        lines.append(f"轮次 {task.get('epoch', 0)}/{task['epochs']}；批次 {task.get('batch', 0)
    }/{task.get('batches', '?')}。")
    if task.get("loss") is not None:
        lines.append(f"最近损失 {task['loss']:.4f}，学习率 {task['lr']:.6g}。")
    if task.get("gpu_memory_mib") is not None:
        lines.append(f"峰值显存 {task['gpu_memory_mib']:.0f} MiB。")
    if task.get("frozen_tensors"):
        lines.append(f"冻结检查：{task['frozen_tensors']} 个旧分支/共享张量保持不变；旧类效果另
    行评估。")
    if task["state"] == "stop_requested":
        lines.append("已请求停止：当前训练轮完成评估并保存后退出；准备阶段在下一个安全点退出。"
    )
    if task.get("result_version_id"):
        lines.append(f"新模型版本：`{task['result_version_id']}`。到【评估与版本】刷新列表后查
    看并选择检测版本。")
    if task.get("error"):
        lines += ["", "**错误：** " + task["error"]]
    chart = []
    for event in store.events(task_id, limit=10000):
        if event["stage"] == "本轮已保存":
            for field, name in (("old_ap", "旧类"), ("new_ap", "新类"), ("overall_ap", "整体"))
    :
                value = event["metrics"].get(field)
                if value is not None:
                    chart.append({"epoch": event["epoch"], "AP50": value, "类别组": name})
    folder = Path(task.get("task_dir", store.root))
    log_path = folder / "console.log"
    log = ""
    if log_path.exists():
        with log_path.open("rb") as stream:
            stream.seek(max(0, log_path.stat().st_size - 14000))
            log = stream.read().decode("utf-8", errors="replace")
    files = [str(folder / name) for name in ("preflight.json", "task_config.json", "dataset_man
    ifest.json",
                                             "baseline_metrics.json", "comparison.json", "console
    .log") if (folder / name).exists()]
    return "\n\n".join(lines), pd.DataFrame(chart, columns=["epoch", "AP50", "类别组"]), log, f
    iles
def progress_figure(chart):
    """Render a complete figure even when no training epochs exist yet.
    Gradio 4.44's native LinePlot mounts listeners on a missing chart element
    with an empty initial value, aborting frontend event initialization.
    """
    import plotly.graph_objects as go
    figure = go.Figure()
```

```
    for name, color in (("旧类", "#2563eb"), ("新类", "#ea580c"), ("整体", "#16a34a")):
        rows = chart[chart["类别组"] == name]
        if not rows.empty:
            figure.add_trace(go.Scatter(x=rows["epoch"].tolist(), y=rows["AP50"].tolist(),
                                        name=name, mode="lines+markers", line={"color": color})
    )
    if chart.empty:
        figure.add_annotation(text="完成一轮评估后显示真实 AP50 曲线", x=.5, y=.5,
                              xref="paper", yref="paper", showarrow=False)
    figure.update_layout(title="逐轮验证 COCO AP50", height=290, margin={"l": 50, "r": 20, "t":
     45, "b": 40},
                         xaxis_title="训练轮次", yaxis_title="AP50", yaxis_range=[0, 1],
                         xaxis={"dtick": 1}, legend={"orientation": "h"}, template="plotly_whit
    e")
    return figure
def monitor_view(store, task_id):
    summary, chart, log, files = monitor_data(store, task_id)
    return summary, progress_figure(chart), log, files
def version_data(store, version_id):
    if not version_id:
        return "请选择模型版本。", [], []
    version = store.version(version_id)
    parent = store.version(version["parent_version_id"]) if version.get("parent_version_id") el
    se None
    info = [f"**{version['name']}** · `{version['id']}`", "",
            "支持类别：" + " / ".join(version["class_names"]), "",
            f"版本关系：{parent['name'] + ' → ' + version['name'] if parent else '外部导入；历
    史父版本未知'}", "",
            f"SHA-256：`{version['sha256']}`"]
    rows = []
    if version.get("task_id"):
        task = store.task(version["task_id"])
        comparison = task.get("comparison", {})
        if comparison:
            before, after = comparison["before"], comparison["after"]
            for index, name in enumerate(CLASSES):
                key = str(index)
                is_new = index in version["new_class_ids"]
                a, b = before["per_class_ap50"].get(key), after["per_class_ap50"].get(key)
                rows.append([name, "新增" if is_new else "旧类", "未支持" if is_new else a, b,
                             None if is_new or a is None or b is None else b-a])
            rows += [["旧类均值", "", before["old_ap"], after["old_ap"], comparison["old_ap_cha
    nge"]],
                     ["新类均值", "新增", "未支持", after["new_ap"], None],
                     ["整体均值", "六类统一评估", before["overall_ap"], after["overall_ap"],
                      after["overall_ap"]-before["overall_ap"]]]
            info += ["", f"选中第 {comparison['selected_epoch']} 轮最佳整体权重；验证图片 {afte
    r['validation_images']} 张。",
                     "协议：COCO AP50，按类 NMS@0.5，全局 top-100。基座整体六类评估将未支持类别
    按无预测计分。"]
```

```
    folder = store.root / "models" / version_id
    files = [str(p) for p in sorted(folder.glob("*.json"))]
    return "\n".join(info), rows, files
def build_workflow_ui(app):
    from util.gradio_compat import WorkflowBlocks, support_boolean_schemas
    support_boolean_schemas()
    store = app.workflow_store()
    recover_interrupted(store)
    base_choices = parent_choices(store, app.BASE_CKPT)
    versions = model_choices(store)
    tasks = task_choices(store)
    selected_base = base_choices[0][1] if base_choices else None
    selected_version = versions[-1][1] if versions else None
    selected_task = tasks[0][1] if tasks else None
    with WorkflowBlocks(title="钢材表面缺陷类增量学习系统") as demo:
        gr.Markdown("# 钢材表面缺陷类增量学习系统\n\n"
                    "演示数据：每类 5 张训练图 · 正式训练请接入完整数据\n\n项目：NEU-DET · 导入
    基座 → 配置新增类别与数据 → 训练 → 对比评估 → 保存并使用新版本")
        with gr.Tabs(selected="base", elem_id="workflow-tabs") as workflow_tabs:
            with gr.Tab("项目与基座", id="base"):
                gr.Markdown("### 1. 初始模型接入\n\n适配器：本项目 Deformable-DETR。支持旧三类
    基座的一次增量及已有模型检测。"
                            "模型先在原训练工程完成初训；此版本通过导入接入，界面尚未提供基座初
    训。"
                            "\n\n本系统已包含该模型家族的结构和增量训练代码，兼容权重无需另交初
    训脚本。"
                            "其他架构仍需提供模型定义并开发适配器，不能仅凭任意权重文件通用训练
    。")
                with gr.Row():
                    with gr.Column():
                        import_name = gr.Textbox(value="旧三类基座 V0", label="模型版本名称")
                        import_path = gr.Textbox(value=str(app.BASE_CKPT), label="已有权重路径"
    )
                        import_file = gr.File(type="filepath", file_types=[".pth"], label="或上
    传权重（优先使用上传文件）")
                        import_config = gr.File(type="filepath", file_types=[".json"], label="
    可选：原模型结构配置 JSON")
                        import_classes = gr.Textbox(label="模型输出列的类别顺序（逗号分隔；元数
    据完整可留空）",
                                                    placeholder="crazing,inclusion,pitted_surfa
    ce")
                        check_button = gr.Button("检查兼容性")
                        register_button = gr.Button("检查并登记模型版本", variant="primary")
                    with gr.Column():
                        import_report = gr.Markdown("等待检查。检查包含严格加载、类别校验和真实
    试推理。", elem_id="model-inspection")
                        gr.Markdown("新数据中的类别编号可在下一页映射。原工程需提供模型结构、类
    别顺序与预处理约定；"
                                    "未知模型家族需要开发对应适配器。")
                next_task = gr.Button("下一步：配置增量任务", interactive=bool(base_choices))
```

```
        with gr.Tab("创建增量任务", id="task"):
            gr.Markdown("### 2. 选择父模型并准备数据\n\n 当前任务类型：类别增量。旧类由父版
本确定，新增类别由兼容 split 确定。")
            parent = gr.Dropdown(choices=base_choices, value=selected_base, allow_custom_va
lue=True,
                                 label="父版本（旧类基座）", info="选择已登记版本或随附基座
；也可粘贴旧三类 .pth 路径并按 Enter 确认。",
                                 elem_id="base-version")
            parent_info = gr.Markdown(parent_description(store, selected_base), elem_id="pa
rent-description")
            with gr.Row():
                check_parent = gr.Button("检查并接入父模型")
                refresh_bases = gr.Button("刷新基座列表", size="sm")
            task_name = gr.Textbox(value="新三类增量训练", label="任务名称")
            source_mode = gr.Radio(["本地文件夹", "上传 ZIP"], value="本地文件夹", label="
新类数据来源")
            new_folder = gr.Textbox(value=str(app.NEU_DET / "train"), label="新类候选数据目
录（images/ + labels/）")
            new_zip = gr.File(type="filepath", file_types=[".zip"], visible=False, label="
新类数据 ZIP", elem_id="new-data-zip")
            class_names = gr.Textbox(value=",".join(CLASSES), label="新数据类别表：按 YOLO
0、1、2…顺序填写名称",
                                     info="若上传数据只有新三类且编号为 0/1/2，填写 patches
,rolled-in_scale,scratches。")
            with gr.Accordion("回放与验证数据来源（标注使用项目全局六类 ID）", open=True):
                replay_source = gr.Textbox(value=str(app.NEU_DET / "train"), label="旧类回
放候选目录")
                validation_source = gr.Textbox(value=str(app.NEU_DET / "valid"), label="独
立验证目录")
                ignore_generated = gr.Checkbox(value=True, label="校验时排除文件名以 _detec
t / _detections 结尾的预览产物，并列入报告")
                exclude_conflicts = gr.Checkbox(value=True, label="排除训练候选中像素相同但
标注冲突的整组图片，并列入报告")
            with gr.Row():
                samples = gr.Number(value=5, minimum=1, maximum=100000, precision=0, label=
"每个新类固定抽取额度")
                replay_count = gr.Number(value=5, minimum=0, maximum=100000, precision=0, l
abel="每个旧类回放额度")
                seed = gr.Number(value=42, minimum=0, maximum=2**31-1, precision=0, label="
固定随机种子")
            with gr.Row():
                epochs = gr.Number(value=1, minimum=1, maximum=1000, precision=0, label="增
量训练轮数")
                batch = gr.Number(value=2, minimum=1, maximum=32, precision=0, label="Batch
 size")
                device = gr.Radio(["cuda", "cpu"], value="cuda" if app.torch.cuda.is_availa
ble() else "cpu", label="训练设备")
            gr.Markdown("采用已验证的 dual 默认策略。新图和纯旧类回放分开保存，训练直接读取
校验后的固定清单；原始数据保留。")
            prepare_button = gr.Button("校验数据并保存训练方案", variant="primary")
```

```
            validation_report = gr.Markdown("等待数据校验。", elem_id="data-report")
            prepared_task = gr.Textbox(label="已保存的任务 ID", interactive=False)
            start_button = gr.Button("启动已校验任务并查看进度", variant="primary", interac
tive=False)
            start_status = gr.Markdown()
        with gr.Tab("训练监控", id="monitor"):
            gr.Markdown("### 3. 真实训练进度\n\n 数据准备 → 基座评估 → 回放准备 → 分支初始化
 → 增量训练 → 最佳模型复核 → 版本保存")
            task_select = gr.Dropdown(choices=tasks, value=selected_task, label="任务历史（
状态持久保存）", elem_id="task-history")
            with gr.Row():
                refresh_tasks = gr.Button("刷新任务列表")
                start_selected = gr.Button("启动所选待运行任务")
                stop_button = gr.Button("请求安全停止")
            monitor_summary = gr.Markdown("选择任务后显示进度。", elem_id="training-summary
")
            curve = gr.Plot(value=progress_figure(pd.DataFrame(columns=["epoch", "AP50", "
类别组"])),
                            label="逐轮验证 COCO AP50", elem_id="training-curve")
            console = gr.Textbox(label="控制台日志（最近部分）", lines=14, interactive=Fals
e)
            task_files = gr.File(file_count="multiple", interactive=False, label="下载任务
配置、清单与结果", elem_id="task-downloads")
            gr.Markdown("关闭浏览器或界面不会停止子进程；重新打开可查看任务。当前版本保留中
断存档，但不提供续训。")
            next_version = gr.Button("查看所选任务的评估与版本")
        with gr.Tab("评估与版本", id="versions"):
            gr.Markdown("### 4. 新旧类对比与模型谱系")
            version_select = gr.Dropdown(choices=versions, value=selected_version, label="
模型版本", elem_id="model-version")
            refresh_versions = gr.Button("刷新版本列表")
            version_summary = gr.Markdown(elem_id="version-summary")
            metrics_table = gr.Dataframe(headers=["类别/指标", "角色", "V0 AP50", "V1 AP50"
, "旧类变化"], interactive=False, elem_id="version-metrics")
            package_files = gr.File(file_count="multiple", interactive=False, label="版本记
录与适配配置（权重由清单路径和哈希引用）", elem_id="version-downloads")
            activate_button = gr.Button("设为日常检测版本", variant="primary")
            activation_status = gr.Markdown(elem_id="activation-status")
            next_detection = gr.Button("前往日常缺陷检测")
            with gr.Accordion("同一张图比较父版本与当前版本", open=False):
                compare_image = gr.Image(type="filepath", label="对比图片", elem_id="compar
ison-upload")
                compare_threshold = gr.Slider(.01, .95, value=.2, step=.01, label="两侧统一
置信度阈值")
                compare_button = gr.Button("比较 V0 / V1")
                with gr.Row():
                    before_image = gr.Image(label="父版本 V0", interactive=False, elem_id="
comparison-before")
                    after_image = gr.Image(label="当前版本 V1", interactive=False, elem_id=
"comparison-after")
```

```
                compare_table = gr.Dataframe(headers=["版本", "类别", "置信度", "x1", "y1",
 "x2", "y2"], interactive=False)
        with gr.Tab("缺陷检测", id="detect"):
            gr.Markdown("### 5. 使用模型检测\n\n 检测阈值仅影响展示；评估与版本中的 AP50 使
用固定协议。")
            ck = gr.Dropdown(choices=app.checkpoint_choices(), value=app.default_checkpoint
(app.list_checkpoints()), label="检测模型", elem_id="detector-model")
            refresh_ck = gr.Button("刷新模型列表", size="sm")
            model_info = gr.Markdown(app.checkpoint_description(app.default_checkpoint(app.
list_checkpoints())))
            with gr.Row():
                with gr.Column():
                    image_input = gr.Image(type="filepath", label="上传缺陷图片", elem_id="
detector-upload")
                    confidence = gr.Slider(.01, .95, value=app.inference_preferences().get(
"confidence_threshold", .2), step=.01, label="置信度阈值")
                    max_detections = gr.Slider(1, 100, value=20, step=1, label="最多显示框
数")
                    infer_button = gr.Button("运行推理", variant="primary")
                with gr.Column():
                    image_output = gr.Image(label="检测结果", interactive=False, elem_id="d
etector-output")
                    detection_table = gr.Dataframe(headers=["类别", "置信度", "x1", "y1", "
x2", "y2"], interactive=False, elem_id="detector-table")
    def chosen_path(path, uploaded):
        selected = file_path(uploaded) or (path or "").strip()
        if not selected:
            raise gr.Error("请选择权重文件或填写路径。")
        return str(app.checkpoint_path(selected))
    @user_errors
    def inspect(path, uploaded, names, config):
        with app.COMPUTE_LOCK:
            app.ensure_compute_available()
            info = inspect_checkpoint(chosen_path(path, uploaded), names, file_path(config)
)
        return inspection_markdown(info)
    @user_errors
    def register(path, uploaded, names, config, name):
        import shutil
        with app.COMPUTE_LOCK:
            app.ensure_compute_available()
            path = chosen_path(path, uploaded)
            if uploaded:
                from services.workflow_store import digest
                target = store.root / "imports" / digest(path) / "weights.pth"
                target.parent.mkdir(parents=True, exist_ok=True)
                if not target.exists():
                    shutil.copy2(path, target)
                path = str(target)
            version = register_model(store, path, name, names, file_path(config))
```

```
        bases = parent_choices(store, app.BASE_CKPT)
        base = version["id"] if version["capabilities"]["incremental_base"] else (bases[0][
1] if bases else None)
        return (inspection_markdown(version) + f"\n\n 已登记版本：`{version['id']}`。", gr.u
pdate(choices=bases, value=base),
                parent_description(store, base), gr.update(choices=model_choices(store), va
lue=version["id"]),
                gr.update(choices=app.checkpoint_choices()), gr.skip(), gr.update(interacti
ve=bool(bases)))
    @user_errors
    def connect_parent(selection):
        with app.COMPUTE_LOCK:
            app.ensure_compute_available()
            version = resolve_parent_model(store, selection, app.PROJECT)
        return (gr.update(choices=parent_choices(store, app.BASE_CKPT), value=version["id"]
),
                parent_description(store, version["id"]), gr.update(interactive=True),
                gr.update(choices=model_choices(store), value=version["id"]),
                gr.update(choices=app.checkpoint_choices()))
    def refresh_parent(current):
        choices = parent_choices(store, app.BASE_CKPT)
        selected = current or (choices[0][1] if choices else None)
        return gr.update(choices=choices, value=selected), parent_description(store, select
ed)
    @user_errors
    def prepare(parent_id, name, mode, folder, uploaded, names, replay, validation, cap, re
play_cap,
                task_seed, task_epochs, task_batch, task_device, ignore, exclude):
        source = file_path(uploaded) if mode == "上传 ZIP" else folder
        if not source:
            raise gr.Error("请选择新类数据。")
        with app.COMPUTE_LOCK:
            app.ensure_compute_available()
            version = resolve_parent_model(store, parent_id, app.PROJECT)
        task = create_incremental_task(store, version["id"], name, source, names, replay, v
alidation,
                                       cap, replay_cap, task_seed, task_epochs, task_batch,
 task_device, ignore, exclude)
        return (preflight_markdown(store, task), task["id"], gr.update(choices=task_choices
(store), value=task["id"]),
                gr.update(interactive=task["state"] == "ready"),
                gr.update(choices=parent_choices(store, app.BASE_CKPT), value=version["id"]
),
                parent_description(store, version["id"]), gr.update(interactive=True),
                gr.update(choices=model_choices(store)), gr.update(choices=app.checkpoint_c
hoices()))
    @user_errors
    def start(task_id):
        if not task_id:
            raise gr.Error("请先校验并保存任务。")
```

```
        with app.COMPUTE_LOCK:
            app.clear_model_cache()
            task = launch_task(store, task_id, app.PROJECT)
        return (f"已启动任务 `{task['id']}`，进程 {task['pid']}。正在【训练监控】显示阶段与
指标。",
                gr.update(choices=task_choices(store), value=task["id"]), gr.update(selecte
d="monitor"))
    def refresh_history(current):
        recover_interrupted(store)
        choices = task_choices(store)
        selected = current if any(value == current for _, value in choices) else choices[0]
[1] if choices else None
        return gr.update(choices=choices, value=selected)
    @user_errors
    def stop(task_id):
        if not task_id:
            raise ValueError("请选择正在运行的任务。")
        store.request_stop(task_id)
        return "已请求安全停止，等待当前训练轮完成评估并保存。"
    def refresh_model_versions(current):
        choices = model_choices(store)
        selected = current if any(value == current for _, value in choices) else choices[-1
][1] if choices else None
        return gr.update(choices=choices, value=selected)
    def load_workflow():
        # Blocks' startup config is cached by the server. Each new browser
        # session must restore records created since build_ui() was called.
        recover_interrupted(store)
        bases, history, registered = parent_choices(store, app.BASE_CKPT), task_choices(sto
re), model_choices(store)
        base = bases[0][1] if bases else None
        active = store.setting("active_version_id")
        version = active if any(value == active for _, value in registered) else registered
[-1][1] if registered else None
        checkpoint = app.default_checkpoint(app.list_checkpoints())
        return (gr.update(choices=bases, value=base), parent_description(store, base),
                gr.update(choices=history, value=history[0][1] if history else None),
                gr.update(choices=registered, value=version),
                gr.update(choices=app.checkpoint_choices(), value=checkpoint), app.checkpoi
nt_description(checkpoint),
                gr.update(interactive=bool(bases)))
    @user_errors
    def task_version(task_id):
        task = store.task(task_id) if task_id else None
        selected = task.get("result_version_id") if task else None
        if not selected:
            raise ValueError("所选任务尚未生成已复核的新版本，请先等待训练完成。")
        return gr.update(choices=model_choices(store), value=selected), gr.update(selected=
"versions")
    @user_errors
```

```
    def activate(version_id):
        if not version_id:
            raise ValueError("请先选择已登记的模型版本。")
        version = activate_version(store, version_id)
        path = Path(version["checkpoint"])
        value = path.relative_to(app.PROJECT).as_posix() if path.is_relative_to(app.PROJECT
) else str(path)
        return f"日常检测版本已设为 {version['name']}，下次打开仍会选择此版本。", gr.update
(choices=app.checkpoint_choices(), value=value), app.checkpoint_description(value)
    def compare(version_id, image, threshold):
        if not version_id:
            raise gr.Error("请选择版本。")
        version = store.version(version_id)
        if not version.get("parent_version_id"):
            raise gr.Error("该导入版本没有已确认的父版本关系，请选择本系统增量训练产生的版
本。")
        if image is None:
            raise gr.Error("请上传对比图片。")
        parent_version = store.version(version["parent_version_id"])
        with app.COMPUTE_LOCK:
            before, a = app.run_inference(parent_version["checkpoint"], image, threshold, 1
00)
            after, b = app.run_inference(version["checkpoint"], image, threshold, 100)
        return before, after, [["V0", *row] for row in a] + [["V1", *row] for row in b]
    check_button.click(inspect, [import_path, import_file, import_classes, import_config],
import_report, api_name="inspect_model")
    register_button.click(register, [import_path, import_file, import_classes, import_confi
g, import_name],
                          [import_report, parent, parent_info, version_select, ck, model_in
fo, next_task], api_name="register_model")
    next_task.click(lambda: gr.update(selected="task"), outputs=workflow_tabs, queue=False,
 api_name=False)
    parent.change(lambda value: parent_description(store, value), parent, parent_info)
    check_parent.click(connect_parent, parent, [parent, parent_info, next_task, version_sel
ect, ck], api_name="connect_parent")
    refresh_bases.click(refresh_parent, parent, [parent, parent_info])
    source_mode.change(lambda mode: (gr.update(visible=mode == "本地文件夹"), gr.update(vis
ible=mode == "上传 ZIP")),
                       source_mode, [new_folder, new_zip])
    prepare_button.click(prepare, [parent, task_name, source_mode, new_folder, new_zip, cla
ss_names, replay_source,
                                   validation_source, samples, replay_count, seed, epochs, b
atch, device,
                                   ignore_generated, exclude_conflicts],
                         [validation_report, prepared_task, task_select, start_button, pare
nt, parent_info, next_task, version_select, ck],
                         api_name="prepare_incremental")
    start_button.click(start, prepared_task, [start_status, task_select, workflow_tabs], ap
i_name="start_incremental")
    start_selected.click(start, task_select, [monitor_summary, task_select, workflow_tabs],
```

```
     api_name="start_selected_task")
        refresh_tasks.click(refresh_history, task_select, task_select)
        stop_button.click(stop, task_select, monitor_summary, api_name="stop_incremental")
        monitor_outputs = [monitor_summary, curve, console, task_files]
        task_select.change(lambda value: monitor_view(store, value), task_select, monitor_outpu
    ts)
        timer = gr.Timer(3)
        timer.tick(lambda value: monitor_view(store, value), task_select, monitor_outputs, show
    _progress="hidden", queue=False)
        next_version.click(task_version, task_select, [version_select, workflow_tabs], api_name
    ="view_task_version")
        refresh_versions.click(refresh_model_versions, version_select, version_select)
        version_select.change(lambda value: version_data(store, value), version_select, [versio
    n_summary, metrics_table, package_files])
        activate_button.click(activate, version_select, [activation_status, ck, model_info], ap
    i_name="activate_version")
        next_detection.click(lambda: gr.update(selected="detect"), outputs=workflow_tabs, queue
    =False, api_name=False)
        compare_button.click(compare, [version_select, compare_image, compare_threshold], [befo
    re_image, after_image, compare_table], api_name="compare_versions")
        ck.change(app.checkpoint_description, ck, model_info)
        refresh_ck.click(app.refresh_checkpoints, ck, [ck, model_info])
        infer_button.click(app.run_inference, [ck, image_input, confidence, max_detections], [i
    mage_output, detection_table], api_name="run_inference")
        loaded = demo.load(load_workflow, outputs=[parent, parent_info, task_select, version_se
    lect, ck, model_info, next_task],
                           api_name="load_workflow")
        loaded.then(lambda value: monitor_view(store, value), task_select, monitor_outputs)
        loaded.then(lambda value: version_data(store, value), version_select, [version_summary,
     metrics_table, package_files])
    return demo
# 文件 adapters/deformable_detr.py
"""Adapter for the trained NEU-DET Deformable-DETR family in this repository."""
import json
from pathlib import Path
import numpy as np
import torch
from services.workflow_store import digest
ADAPTER_ID = "neu_deformable_detr_v1"
CLASSES = ["crazing", "inclusion", "patches", "pitted_surface", "rolled-in_scale", "scratches"]
SPLITS = {"1": {"old": [0, 1, 3], "new": [2, 4, 5]},
          "2": {"old": [0, 2, 4], "new": [1, 3, 5]},
          "3": {"old": [1, 3, 5], "new": [0, 2, 4]}}
def inference_args(device=None):
    from incremental_train import get_args_parser
    args = get_args_parser().parse_args([])
    for key, value in {"position_embedding": "sine", "position_embedding_scale": 2 * np.pi,
                       "masks": False, "cache_mode": False, "aux_loss": True,
                       "new_class_cost_discount": .5}.items():
        if not hasattr(args, key):
```

```
            setattr(args, key, value)
    args.device = device or ("cuda" if torch.cuda.is_available() else "cpu")
    return args
def describe_checkpoint(checkpoint, class_names="", config=None):
    saved = checkpoint.get("args", {})
    saved = saved if isinstance(saved, dict) else vars(saved)
    native_metadata = {**saved, **checkpoint.get("model_config", {})}
    metadata = {**native_metadata, **(config or {})}
    state = checkpoint.get("model", checkpoint.get("model_state_dict", checkpoint))
    heads = [v for k, v in state.items() if k.startswith("class_embed.") and k.endswith("weight
    ") and getattr(v, "ndim", 0) == 2]
    if not heads:
        raise ValueError("无法识别分类头；请选择本项目 Deformable-DETR 存档。")
    count = int(heads[0].shape[0])
    if any(int(v.shape[0]) != count for v in heads):
        raise ValueError("分类头层间输出数量不一致。")
    dual = any(k.startswith("new_branch.") for k in state)
    adapted = any(k.startswith("head_adapters.") for k in state)
    supplied = [name.strip() for name in class_names.replace("，", ",").split(",") if name.stri
    p()]
    if supplied and (len(supplied) != count or len(set(supplied)) != count or any(n not in CLAS
    SES for n in supplied)):
        raise ValueError(f"需要提供 {count} 个不重复的已支持类别名，按模型输出列的顺序填写。")
    order = native_metadata.get("class_global_ids", checkpoint.get("phase_classes"))
    source = "存档类别元数据"
    if order is None:
        group = str(native_metadata.get("split_group", metadata.get("split_group", "")))
        if checkpoint.get("phase") == 0 and group in SPLITS and count == len(SPLITS[group]["old
    "]):
            order = SPLITS[group]["old"]
            source = "历史 phase-0 的 split 配置"
        elif "model_state_dict" in checkpoint and count == 6:
            order = list(range(6))
            source = "本项目历史全六类训练格式"
        elif checkpoint.get("phase", 0) > 0 and group in SPLITS and count == 6:
            order = sorted(SPLITS[group]["old"] + SPLITS[group]["new"])
            source = "本项目历史增量阶段的 split 配置"
    config_order = (config or {}).get("class_global_ids")
    if config_order is not None:
        if order is not None and list(order) != list(config_order):
            raise ValueError("伴随配置的类别顺序与存档元数据冲突。")
        order = config_order
        source = "已核对的伴随类别配置"
    if supplied:
        supplied_ids = [CLASSES.index(name) for name in supplied]
        if order is not None and list(order) != supplied_ids:
            raise ValueError("填写的类别顺序与存档元数据冲突，请核对原始训练代码。")
        order = supplied_ids
        source += "；用户已确认" if source else "用户提供"
    if order is None or len(order) != count or len(set(order)) != count or any(i not in range(6
```

```
    ) for i in order):
        raise ValueError("缺少可靠的类别顺序，请按原训练模型输出列填写类别名。")
    order = list(order)
    split = next((key for key, value in SPLITS.items() if value["old"] == order), None)
    phase = int(checkpoint.get("phase", 0))
    can_increment = bool(split and not dual and not adapted and "model_state_dict" not in check
    point and phase == 0)
    return {"adapter_id": ADAPTER_ID, "architecture": "dual" if dual else "legacy_full" if "mod
    el_state_dict" in checkpoint else "adapter" if adapted else "baseline",
            "class_order": order, "class_names": [CLASSES[i] for i in order], "class_source": s
    ource,
            "split_group": split, "stage": 1 if dual else 0,
            "capabilities": {"inference": True, "incremental_base": can_increment,
                             "resume": False, "sequential_increment": False, "base_training_ui"
    : False},
            "limitation": "支持从旧三类基座进行一次增量，暂不支持连续增量或中断续训。" if can_i
    ncrement else "可用于检测；当前适配器不支持以该版本创建类别增量任务。",
            "model_config": metadata, "epoch": checkpoint.get("epoch"),
            "checkpoint_metrics": checkpoint.get("metrics", {})}
def load_model(path, device=None, class_names="", config=None):
    from incremental_train import safe_torch_load
    from models.incremental_detr import build_incremental_model
    from util.checkpoint_config import apply_checkpoint_model_args, load_inference_weights, res
    tore_model_metadata
    checkpoint = safe_torch_load(str(path), map_location="cpu")
    construction = ({**checkpoint, "model_config": {**checkpoint.get("model_config", {}), **con
    fig}}
                    if config else checkpoint)
    info = describe_checkpoint(checkpoint, class_names, config)
    args = inference_args(device)
    if "model_state_dict" in checkpoint:
        from util.checkpoint_config import build_legacy_full_model
        model = build_legacy_full_model(checkpoint)
        model.load_state_dict(checkpoint["model_state_dict"], strict=True)
    else:
        apply_checkpoint_model_args(args, construction)
        model, _ = build_incremental_model(args, num_classes=len(info["class_order"]))
        # Preserve the native format marker: a sidecar must not make a legacy
        # checkpoint look like a new-format checkpoint with mandatory shape keys.
        load_inference_weights(model, checkpoint)
        if config:
            restore_model_metadata(model, construction)
    model.class_global_ids = info["class_order"]
    args.class_global_ids = info["class_order"]
    model.to(args.device).eval()
    return model, args, info
def inspect_checkpoint(path, class_names="", config_path=None):
    config = json.loads(Path(config_path).read_text(encoding="utf-8")) if config_path else None
    if config is not None and not isinstance(config, dict):
        raise ValueError("模型配置 JSON 必须是对象。")
```

```
    model, args, info = load_model(path, device="cpu", class_names=class_names, config=config)
    from util.misc import nested_tensor_from_tensor_list
    # A shape-only inspection cannot validate the native attention path.
    threads = torch.get_num_threads()
    try:
        torch.set_num_threads(min(threads, 4))
        with torch.no_grad():
            output = model(nested_tensor_from_tensor_list([torch.zeros(3, 224, 224)]))
    finally:
        torch.set_num_threads(threads)
    if (output["pred_logits"].shape[-1] != len(info["class_order"]) or
            not torch.isfinite(output["pred_logits"]).all() or not torch.isfinite(output["pred_
    boxes"]).all()):
        raise ValueError("模型试推理输出无效。")
    info.update(checkpoint=str(Path(path).resolve()), sha256=digest(path), verified=True,
                preprocess={"adapter": "NEU_DET_val", "val_size": 224, "mean": [.5, .5, .5], "s
    td": [.5, .5, .5],
                            "postprocess": "perclassNMS@0.5_global_top100"})
    # Store only JSON model construction metadata, not the full historic Namespace.
    from util.checkpoint_config import MODEL_ARGUMENTS, model_config
    if hasattr(model, "set_shape_gate_ids"):
        info["model_config"] = model_config(model, args)
    else:
        metadata = info["model_config"]
        info["model_config"] = {key: metadata.get(key, getattr(args, key, None)) for key in MOD
    EL_ARGUMENTS
                                if key in metadata or hasattr(args, key)}
    return info
@torch.no_grad()
def evaluate(model, loader, device, old_ids, new_ids):
    from models.deformable_detr import PostProcess
    from util.detection_metrics import coco_ap50
    model.eval()
    predictions, targets = [], []
    order = list(model.class_global_ids)
    postprocess = PostProcess(nms_mode="perclass", max_detections=100)
    for samples, batch_targets in loader:
        outputs = model(samples.to(device))
        sizes = torch.stack([target["orig_size"] for target in batch_targets]).to(device)
        results = postprocess(outputs, sizes)
        for result, target in zip(results, batch_targets):
            predictions.append((result["boxes"].cpu().numpy(),
                                np.array([order[i] for i in result["labels"].cpu().tolist()]),
                                result["scores"].cpu().numpy()))
            targets.append(target)
    per_class = coco_ap50(predictions, targets, CLASSES)
    def mean(ids):
        values = [per_class[i] for i in ids if per_class[i] is not None]
        return float(np.mean(values)) if values else None
    return {"metric_protocol": "COCO_AP50_perclassNMS_top100", "per_class_ap50": per_class,
```

```
            "old_ap": mean(old_ids), "new_ap": mean(new_ids), "overall_ap": mean(range(6)),
            "supported_class_ids": order, "validation_images": len(targets)}
# 文件 services/dataset_service.py
"""Validate, map and snapshot the exact data consumed by a training task."""
import hashlib
import io
import math
import random
import zipfile
from collections import Counter
from pathlib import Path, PurePosixPath
from PIL import Image
from adapters.deformable_detr import CLASSES, SPLITS
from services.workflow_store import digest, write_json
IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp"}
def label_mapping(names):
    if isinstance(names, str):
        names = [name.strip() for name in names.replace("，", ",").split(",") if name.strip()]
    if not names or len(set(names)) != len(names) or any(name not in CLASSES for name in names)
    :
        raise ValueError("请按数据集标注 ID 顺序填写不重复的类别名称；当前适配器支持 NEU-DET 六
    类。")
    return {index: CLASSES.index(name) for index, name in enumerate(names)}
def parse_labels(text, mapping, filename):
    boxes = []
    for line_no, line in enumerate(text.splitlines(), 1):
        if not line.strip():
            continue
        parts = line.split()
        try:
            if len(parts) != 5:
                raise ValueError("每行应为 cls cx cy w h 共 5 列")
            class_id = int(parts[0])
            if class_id not in mapping:
                raise ValueError(f"标注类别 {class_id} 没有映射")
            cx, cy, width, height = map(float, parts[1:])
            if not all(math.isfinite(v) for v in (cx, cy, width, height)):
                raise ValueError("框含非有限数值")
            if width <= 0 or height <= 0:
                raise ValueError("框宽高必须大于零")
            if min(cx-width/2, cy-height/2) < -1e-6 or max(cx+width/2, cy+height/2) > 1+1e-6:
                raise ValueError("归一化框超出图片边界")
            boxes.append([mapping[class_id], cx, cy, width, height])
        except ValueError as exc:
            raise ValueError(f"{filename}:{line_no}：{exc}") from exc
    return boxes
def _pair_key(name, directory):
    parts = list(PurePosixPath(name.replace("\\", "/")).parts)
    if directory in parts:
        parts.remove(directory)
```

```
    return str(PurePosixPath(*parts).with_suffix("")).casefold()
def scan_source(source, mapping, ignore_generated=False):
    source = Path(source).resolve()
    records, errors, warnings = [], [], []
    archive = None
    try:
        if source.is_file() and source.suffix.lower() == ".zip":
            archive = zipfile.ZipFile(source)
            names = [entry.filename for entry in archive.infolist() if not entry.is_dir()]
            if len(names) != len(set(names)):
                raise ValueError("ZIP 中存在重复路径。")
            def read(name):
                return archive.read(name)
        elif source.is_dir():
            names = [p.relative_to(source).as_posix() for p in sorted(source.rglob("*")) if p.i
    s_file()]
            def read(name):
                return (source / name).read_bytes()
        else:
            raise ValueError(f"数据源不存在或不是目录/ZIP：{source}")
        images, labels = {}, {}
        for name in names:
            suffix = Path(name).suffix.lower()
            is_image = suffix in IMAGE_EXTENSIONS
            if not is_image and (suffix != ".txt" or Path(name).name.lower() in ("classes.txt",
     "readme.txt")):
                continue
            if is_image and ignore_generated and Path(name).stem.endswith(("_detections", "_det
    ect")):
                warnings.append(f"排除预览产物：{name}")
                continue
            table, kind = (images, "images") if is_image else (labels, "labels")
            key = _pair_key(name, kind)
            if key in table:
                errors.append(f"存在歧义的同名文件：{table[key]} / {name}")
            table[key] = name
        for key, name in sorted(images.items()):
            label = labels.get(key)
            if label is None:
                errors.append(f"缺少标注：{name}（有效负样本请提供空 TXT）")
                continue
            try:
                raw = read(name)
                with Image.open(io.BytesIO(raw)) as im:
                    image = im.convert("RGB")
                    image.load()
                    width, height = image.size
                    pixels = hashlib.sha256(str(image.size).encode() + image.tobytes()).hexdige
    st()
                raw_label = read(label)
```

```
                text = raw_label.decode("utf-8-sig")
                boxes = parse_labels(text, mapping, label)
                records.append({"source": str(source), "image": name, "label": label, "width":
    width,
                                "height": height, "sha256": hashlib.sha256(raw).hexdigest(),
                                "pixel_sha256": pixels, "label_sha256": hashlib.sha256(raw_labe
    l).hexdigest(),
                                "boxes": boxes, "classes": sorted({row[0] for row in boxes})})
            except Exception as exc:
                errors.append(f"{name}: {exc}")
        if not images:
            errors.append("数据源中没有可用图片。")
        orphan_count = len(set(labels) - set(images))
        if orphan_count:
            warnings.append(f"有 {orphan_count} 个标注没有对应图片。")
    except Exception as exc:
        errors.append(str(exc))
    finally:
        if archive is not None:
            archive.close()
    return {"source": str(source), "records": records, "errors": errors, "warnings": warnings}
def _deduplicate(records, errors, warnings, role, exclude_conflicts=False):
    unique, conflicts = {}, set()
    for record in records:
        key = record["pixel_sha256"]
        if key in unique:
            previous = unique[key]
            if sorted(previous["boxes"]) != sorted(record["boxes"]):
                message = f"{role}相同图片有不同标注：{previous['image']} / {record['image']}"
                if exclude_conflicts:
                    warnings.append("排除冲突图片组：" + message)
                    conflicts.add(key)
                else:
                    errors.append(message)
            else:
                warnings.append(f"{role}去除重复图片：{record['image']}")
        else:
            unique[key] = record
    return [record for key, record in unique.items() if key not in conflicts]
def _sample(records, class_ids, count, seed):
    rng = random.Random(seed)
    selected, draws = set(), {}
    for class_id in class_ids:
        candidates = [index for index, record in enumerate(records) if class_id in record["clas
    ses"]]
        chosen = rng.sample(candidates, min(count, len(candidates)))
        draws[str(class_id)] = len(chosen)
        selected.update(chosen)
    return [records[i] for i in sorted(selected)], draws
def _counts(records):
```

```
    images, boxes = Counter(), Counter()
    for record in records:
        images.update(record["classes"])
        boxes.update(row[0] for row in record["boxes"])
    return {str(i): {"images": images[i], "boxes": boxes[i]} for i in range(6)}
def preflight(new_source, class_names, replay_source, validation_source, split_group,
              samples=100, replay_per_class=50, seed=42, ignore_generated=False, exclude_confli
    cts=False):
    if split_group not in SPLITS:
        raise ValueError("不支持的类别划分。")
    if not 1 <= int(samples) <= 100000 or not 0 <= int(replay_per_class) <= 100000:
        raise ValueError("每类样本额度需为正数；回放额度可为零。")
    mapping = label_mapping(class_names)
    old, new = SPLITS[split_group]["old"], SPLITS[split_group]["new"]
    sources = {"new": scan_source(new_source, mapping),
               "replay": scan_source(replay_source, dict(enumerate(range(6)))),
               "validation": scan_source(validation_source, dict(enumerate(range(6))), ignore_g
    enerated)}
    errors, warnings = [], []
    for role, source in sources.items():
        errors.extend(f"[{role}] {message}" for message in source["errors"])
        warnings.extend(f"[{role}] {message}" for message in source["warnings"])
        source["records"] = _deduplicate(source["records"], errors, warnings, role,
                                         exclude_conflicts and role != "validation")
    new_candidates = [record for record in sources["new"]["records"] if set(record["classes"])
    & set(new)]
    replay_candidates = [record for record in sources["replay"]["records"]
                         if record["classes"] and set(record["classes"]) <= set(old)]
    warnings.append(f"新增数据仅采用含本次新类的图片：{len(new_candidates)} 张；旧类回放候选限
    定纯旧类：{len(replay_candidates)} 张。")
    chosen_new, new_draws = _sample(new_candidates, new, int(samples), int(seed))
    chosen_replay, replay_draws = _sample(replay_candidates, old, int(replay_per_class), int(se
    ed)+1)
    validation = sources["validation"]["records"]
    for class_id in new:
        if new_draws[str(class_id)] == 0:
            errors.append(f"新增类别 {CLASSES[class_id]} 没有可训练图片。")
        elif new_draws[str(class_id)] < int(samples):
            warnings.append(f"{CLASSES[class_id]} 不足 {samples} 张，实际抽取 {new_draws[str(cl
    ass_id)]} 张。")
    for class_id in old:
        if replay_draws[str(class_id)] < int(replay_per_class):
            warnings.append(f"旧类 {CLASSES[class_id]} 回放不足额度，实际抽取 {replay_draws[str
    (class_id)]} 张。")
    val_counts = _counts(validation)
    for class_id in range(6):
        if not val_counts[str(class_id)]["images"]:
            errors.append(f"验证集缺少 {CLASSES[class_id]} 标注，当前完整增量评估需要全部六类。
    ")
    training_hashes = {record["pixel_sha256"] for record in chosen_new+chosen_replay}
```

```
    for record in validation:
        if record["pixel_sha256"] in training_hashes:
            errors.append(f"训练/回放与验证图片重复：{record['image']}")
    manifest = {"schema_version": 1, "split_group": split_group, "old_class_ids": old, "new_cla
    ss_ids": new,
                "label_mapping": mapping, "sample_seed": int(seed), "samples_per_new_class": in
    t(samples),
                "replay_per_old_class": int(replay_per_class), "sampling_policy": "seeded_rando
    m_per_class_union",
                "new_draws": new_draws, "replay_draws": replay_draws,
                "new": chosen_new, "replay": chosen_replay, "validation": validation,
                "counts": {"new": _counts(chosen_new), "replay": _counts(chosen_replay), "valid
    ation": val_counts},
                "sources": {key: value["source"] for key, value in sources.items()},
                "exclude_conflicting_training_images": bool(exclude_conflicts),
                "warnings": warnings, "errors": errors}
    return manifest
def snapshot(manifest, task_dir):
    if manifest["errors"]:
        raise ValueError("数据未通过校验，不能生成训练清单。")
    task_dir = Path(task_dir)
    for role in ("new", "replay", "validation"):
        folder = task_dir / "dataset" / role
        (folder / "images").mkdir(parents=True, exist_ok=True)
        (folder / "labels").mkdir(exist_ok=True)
        for index, record in enumerate(manifest[role]):
            source = Path(record["source"])
            if source.is_file():
                with zipfile.ZipFile(source) as archive:
                    raw = archive.read(record["image"])
                    label_raw = archive.read(record["label"])
            else:
                raw = (source / record["image"]).read_bytes()
                label_raw = (source / record["label"]).read_bytes()
            if hashlib.sha256(raw).hexdigest() != record["sha256"] or hashlib.sha256(label_raw)
    .hexdigest() != record["label_sha256"]:
                raise ValueError(f"校验后源文件发生变化，请重新检查：{record['image']}")
            stem = f"{index:06d}_{record['sha256'][:12]}"
            image_name = stem + Path(record["image"]).suffix.lower()
            label_name = stem + ".txt"
            (folder / "images" / image_name).write_bytes(raw)
            text = "".join(f"{int(row[0])} " + " ".join(f"{v:.10g}" for v in row[1:]) + "\n" fo
    r row in record["boxes"])
            (folder / "labels" / label_name).write_text(text, encoding="utf-8")
            record["staged_image"] = f"dataset/{role}/images/{image_name}"
            record["staged_label"] = f"dataset/{role}/labels/{label_name}"
            record["staged_label_sha256"] = digest(folder / "labels" / label_name)
    path = task_dir / "dataset_manifest.json"
    write_json(path, manifest)
    return path
```

```
def verify_snapshot(task_dir, manifest):
    root = Path(task_dir).resolve()
    for role in ("new", "replay", "validation"):
        expected_images, expected_labels = set(), set()
        for record in manifest[role]:
            image, label = root / record["staged_image"], root / record["staged_label"]
            if not image.resolve().is_relative_to(root) or not label.resolve().is_relative_to(r
    oot):
                raise ValueError("快照路径超出任务目录。")
            if digest(image) != record["sha256"] or digest(label) != record["staged_label_sha25
    6"]:
                raise ValueError(f"任务快照已改变：{record['staged_image']}")
            expected_images.add(image.name)
            expected_labels.add(label.name)
        folder = root / "dataset" / role
        if {p.name for p in (folder / "images").iterdir()} != expected_images or {p.name for p
    in (folder / "labels").iterdir()} != expected_labels:
            raise ValueError(f"任务 {role} 目录有额外或缺失文件。")
    return True
# 文件 services/task_service.py
"""Create immutable task plans and supervise separate training processes."""
import json
import os
import subprocess
import sys
import time
from pathlib import Path
from adapters.deformable_detr import ADAPTER_ID, CLASSES, inspect_checkpoint
from services.dataset_service import preflight, snapshot, verify_snapshot
from services.workflow_store import ACTIVE_STATES, digest, timestamp, write_json
_PROCESSES = {}
def register_model(store, checkpoint, name, class_names="", config_path=None, require_increment
    al_base=False):
    if not name or not name.strip():
        raise ValueError("请填写模型版本名称。")
    info = inspect_checkpoint(checkpoint, class_names, config_path)
    if require_incremental_base and not info["capabilities"].get("incremental_base"):
        raise ValueError("该权重不能作为增量父模型。请选择旧三类 phase-0 基座；六类 dual 模型目
    前只支持检测。")
    if digest(checkpoint) != info["sha256"]:
        raise ValueError("检查期间权重发生变化，请重新导入。")
    info.update(name=name.strip(), project_id="neu-det", parent_version_id=None, task_id=None,
                task_type="imported_base" if info["capabilities"]["incremental_base"] else "imp
    orted_inference",
                provenance={"kind": "import", "training_data": "unknown", "historical_parent":
    "unknown"})
    version = store.register_version(info)
    export_package(store, version)
    return version
def resolve_parent_model(store, selection, project_root):
```

```
    selected = str(selection or "").strip()
    if not selected:
        raise ValueError("请选择父版本，或填写旧三类基座的 .pth 文件路径。")
    version = next((item for item in store.versions() if item["id"] == selected), None)
    if version is not None:
        if version.get("adapter_id") != ADAPTER_ID or not version["capabilities"].get("incremen
    tal_base"):
            raise ValueError("该版本不支持增量训练，请选择旧三类 phase-0 基座。")
        return version
    if selected.startswith("v_"):
        raise ValueError("父版本记录不存在，请刷新基座列表或重新填写基座权重路径。")
    if len(selected) > 1 and selected[0] == selected[-1] and selected[0] in {'"', "'"}:
        selected = selected[1:-1].strip()
    path = Path(os.path.expandvars(selected)).expanduser()
    if not path.is_absolute():
        path = Path(project_root) / path
    path = path.resolve()
    if not path.is_file():
        raise ValueError(f"找不到基座权重文件：{path}。请填写 .pth 文件完整路径，不能填写文件夹
    或模型名称。")
    return register_model(store, path, f"旧类基座 · {path.stem}", require_incremental_base=True
    )
def export_package(store, version):
    folder = store.root / "models" / version["id"]
    # An external large weight remains referenced by absolute path and checksum.
    write_json(folder / "manifest.json", version | {"weight_storage": "external_reference"})
    write_json(folder / "model_config.json", version.get("model_config", {}))
    write_json(folder / "classes.json", {"registry": CLASSES, "class_order": version["class_ord
    er"]})
    write_json(folder / "preprocess.json", version.get("preprocess", {}))
    write_json(folder / "metrics.json", version.get("metrics", version.get("checkpoint_metrics"
    , {})))
    return folder
def create_incremental_task(store, parent_id, name, new_source, class_names, replay_source,
                            validation_source, samples=100, replay_per_class=50, seed=42,
                            epochs=60, batch_size=6, device="cuda", ignore_generated=False, exc
    lude_conflicts=False):
    parent = store.version(parent_id)
    if parent.get("adapter_id") != ADAPTER_ID or not parent["capabilities"].get("incremental_ba
    se"):
        raise ValueError("请选择已登记的旧三类 phase-0 基座；当前不支持从已有 dual 模型再次增量
    。")
    if not name or not name.strip():
        raise ValueError("请填写任务名称。")
    if not 1 <= int(epochs) <= 1000 or not 1 <= int(batch_size) <= 32:
        raise ValueError("训练轮数范围 1～1000，batch 范围 1～32。")
    if device not in ("cuda", "cpu"):
        raise ValueError("设备只支持 cuda 或 cpu。")
    if not 0 <= int(seed) < 2**31:
        raise ValueError("随机种子范围是 0～2147483647。")
```

```
    if digest(parent["checkpoint"]) != parent["sha256"]:
        raise ValueError("基座权重与登记哈希不一致，请重新导入核对。")
    task = store.create_task({"name": name.strip(), "project_id": "neu-det", "task_type": "clas
    s_incremental",
                              "parent_version_id": parent_id, "state": "validating", "stage": "
    数据校验"})
    task_dir = store.root / "tasks" / task["id"]
    task_dir.mkdir(parents=True)
    store.update_task(task["id"], task_dir=str(task_dir))
    try:
        manifest = preflight(new_source, class_names, replay_source, validation_source, parent[
    "split_group"],
                             samples, replay_per_class, seed, ignore_generated, exclude_conflic
    ts)
        write_json(task_dir / "preflight.json", manifest)
        store.event(task["id"], "数据校验", errors=len(manifest["errors"]), warnings=len(manife
    st["warnings"]))
        if manifest["errors"]:
            return store.update_task(task["id"], state="failed", stage="数据校验未通过", error=
    "；".join(manifest["errors"][:10]))
        manifest_path = snapshot(manifest, task_dir)
        config = {"adapter_id": ADAPTER_ID, "parent_version_id": parent_id,
                  "parent_checkpoint": parent["checkpoint"], "parent_sha256": parent["sha256"],
                  "parent_class_names": parent["class_names"], "parent_model_config": parent["m
    odel_config"],
                  "split_group": parent["split_group"], "epochs": int(epochs), "batch_size": in
    t(batch_size),
                  "seed": int(seed), "device": device, "samples": int(samples),
                  "replay_per_class": int(replay_per_class), "manifest_sha256": digest(manifest
    _path),
                  "metric_protocol": "COCO_AP50_perclassNMS_top100", "checkpoint_selection": "b
    est_overall_ap50",
                  "dual_lr": 1e-4, "dual_head_lr": 5e-4, "accum_steps": 2,
                  "dual_dn_groups": 0, "dual_quality_loss": False}
        config_path = task_dir / "task_config.json"
        write_json(config_path, config)
        return store.update_task(task["id"], state="ready", stage="待启动", config=config,
                                 config_sha256=digest(config_path), manifest_sha256=config["ma
    nifest_sha256"],
                                 counts=manifest["counts"], warnings=manifest["warnings"],
                                 unique_images={role: len(manifest[role]) for role in ("new",
    "replay", "validation")})
    except Exception as exc:
        store.update_task(task["id"], state="failed", stage="数据准备失败", error=str(exc))
        raise
def launch_task(store, task_id, project_root):
    recover_interrupted(store)
    task = store.task(task_id)
    if task["state"] != "ready":
        raise ValueError("只能启动已通过校验且尚未运行的任务；修改方案请创建新任务。")
```

```
    task_dir = Path(task["task_dir"])
    config = task["config"]
    if digest(task_dir / "task_config.json") != task["config_sha256"] or digest(task_dir / "dat
    aset_manifest.json") != task["manifest_sha256"]:
        raise ValueError("任务配置或样本清单已变化，请重新创建任务。")
    if digest(config["parent_checkpoint"]) != config["parent_sha256"]:
        raise ValueError("父版本权重已变化，请重新导入。")
    verify_snapshot(task_dir, json.loads((task_dir / "dataset_manifest.json").read_text(encodin
    g="utf-8")))
    store.claim_task(task_id)
    command = [sys.executable, "-u", str(Path(project_root) / "tools" / "run_incremental_task.p
    y"),
               "--store", str(store.root), "--task-id", task_id]
    try:
        env = {**os.environ, "PYTHONIOENCODING": "utf-8", "PYTHONDONTWRITEBYTECODE": "1",
               "GRADIO_ANALYTICS_ENABLED": "False", "NEU_VAL_SIZE": "224"}
        kwargs = {"creationflags": subprocess.CREATE_NO_WINDOW} if os.name == "nt" else {}
        with (task_dir / "console.log").open("ab") as log:
            process = subprocess.Popen(command, cwd=str(project_root), env=env, stdout=log,
                                       stderr=subprocess.STDOUT, **kwargs)
        _PROCESSES[task_id] = process
        return store.update_task(task_id, pid=process.pid, command=command, launched_at=time.ti
    me())
    except Exception as exc:
        store.update_task(task_id, state="failed", stage="启动失败", error=str(exc))
        raise
def _alive(pid):
    if not pid:
        return False
    if os.name == "nt":
        import ctypes
        from ctypes import wintypes
        kernel = ctypes.WinDLL("kernel32", use_last_error=True)
        kernel.OpenProcess.argtypes = [wintypes.DWORD, wintypes.BOOL, wintypes.DWORD]
        kernel.OpenProcess.restype = wintypes.HANDLE
        kernel.GetExitCodeProcess.argtypes = [wintypes.HANDLE, ctypes.POINTER(wintypes.DWORD)]
        kernel.CloseHandle.argtypes = [wintypes.HANDLE]
        handle = kernel.OpenProcess(0x1000, False, int(pid))
        if not handle:
            return False
        try:
            code = wintypes.DWORD()
            return bool(kernel.GetExitCodeProcess(handle, ctypes.byref(code)) and code.value ==
     259)
        finally:
            kernel.CloseHandle(handle)
    try:
        os.kill(int(pid), 0)
        return True
    except ProcessLookupError:
```

```
        return False
    except PermissionError:
        return True
def recover_interrupted(store):
    for task_id, process in list(_PROCESSES.items()):
        if process.poll() is not None:
            _PROCESSES.pop(task_id, None)
    for task in store.tasks():
        if task["state"] not in ACTIVE_STATES:
            continue
        process = _PROCESSES.get(task["id"])
        alive = process.poll() is None if process is not None else _alive(task.get("pid"))
        if not alive and time.time() - task.get("launched_at", time.time()) > 15:
            store.update_task(task["id"], state="interrupted", stage="进程已中断",
                              error="训练进程已退出且未写入完成状态；保留日志及最近存档。当前版
    本不提供续训。")
def active_task(store):
    recover_interrupted(store)
    return next((task for task in store.tasks() if task["state"] in ACTIVE_STATES), None)
def activate_version(store, version_id):
    version = store.version(version_id)
    if not version.get("verified") or not version["capabilities"].get("inference"):
        raise ValueError("该版本未完成模型验证。")
    if digest(version["checkpoint"]) != version["sha256"]:
        raise ValueError("权重已变化，无法设为检测版本。")
    store.set_setting("active_version_id", version_id)
    store.set_setting("activation", {"version_id": version_id, "created": timestamp()})
    return version
# 文件 services/workflow_store.py
"""Small transactional registry; large datasets and weights stay on disk."""
import hashlib
import json
import os
import sqlite3
import time
import uuid
from contextlib import contextmanager
from datetime import datetime, timezone
from pathlib import Path
ACTIVE_STATES = ("running", "evaluating", "stop_requested")
TERMINAL_STATES = ("succeeded", "stopped", "failed", "interrupted")
def timestamp():
    return datetime.now(timezone.utc).isoformat(timespec="seconds")
def digest(path):
    with Path(path).open("rb") as stream:
        return hashlib.file_digest(stream, "sha256").hexdigest()
def write_json(path, value):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_name(path.name + "." + uuid.uuid4().hex + ".tmp")
```

```
            "shape_loss_coef",
            2.0,
        ),
    # v108：长宽比监督（治 H1 根因，arctan 饱和 -> log 空间梯度不衰减）
    "loss_aspect":
        getattr(
            args,
            "aspect_loss_coef",
            2.0,
        ),
}
# ------------------------------------------------------
# Incremental losses
# ------------------------------------------------------
weight_dict[
    "loss_distill"
] = getattr(
    args,
    "distill_weight",
    0.2,
)
weight_dict[
    "loss_feat_distill"
] = getattr(
    args,
    "feat_distill_weight",
    0.5,
)
weight_dict[
    "loss_reg"
] = getattr(
    args,
    "reg_weight",
    0.01,
)
# v81：旧类框蒸馏权重（L1+GIoU 锚定旧类框）
# 20：冒烟测试显示 5.0 时框头漂移仍致旧类 AP -0.03
weight_dict[
    "loss_box_kd"
] = 20.0
# v81c：flip_guard 权重 1.0（原 10.0 → 2.0 → 1.0）。
# 范围已收窄为"匹配到旧类 GT 的 query"且 aux 层不再
# 复制，总权重从 ~4000 降到 ~112，保留防翻转能力，
# 不再碾压新类分类头
weight_dict[
    "loss_flip_guard"
] = 1.0
# 新类 hard negative 默认轻量
weight_dict[
    "loss_hard_neg"
```

```
] = getattr(
    args,
    "hard_neg_loss_coef",
    0.25,
)
# ------------------------------------------------------------
# Aux losses
# ------------------------------------------------------------
if args.aux_loss:
    aux_weight_dict = {}
    for i in range(
        args.dec_layers - 1
    ):
        for k, v in weight_dict.items():
            aux_weight_dict[
                f"{k}_{i}"
            ] = v
    weight_dict.update(
        aux_weight_dict
    )
# ------------------------------------------------------------
# Criterion
# ------------------------------------------------------------
criterion = IncrementalSetCriterion(
    num_classes=num_classes,
    matcher=matcher,
    weight_dict=weight_dict,
    losses=[
        "labels",
        "boxes",
        "cardinality",
    ],
    focal_alpha=args.focal_alpha,
    old_num_classes=0,
    distill_weight=getattr(
        args,
        "distill_weight",
        0.2,
    ),
    feat_distill_weight=getattr(
        args,
        "feat_distill_weight",
        0.5,
    ),
    reg_weight=getattr(
        args,
        "reg_weight",
        0.01,
    ),
    class_weights=class_weights,
```

```
    )
    criterion.to(device)
    return model, criterion
# 文件 dual_branch_train.py
"""Training for the independent new decoder, isolated from legacy shared-head KD."""
import copy
import json
import time
import numpy as np
import torch
from torch.utils.data import ConcatDataset, DataLoader
from datasets.neu_det import NEU_DET_CLASSES, build_neu_det_dataset, build_small_sample_dataset
from models.incremental_detr import build_incremental_model
from util import misc
from util.box_ops import box_cxcywh_to_xyxy
from util.detection_metrics import coco_ap50
@torch.no_grad()
def evaluate_dual(model, loader, device, old_classes, new_classes):
    from util.detection_metrics import postprocess_variant, ap_from_preds
    was_training = model.training
    model.eval()
    predictions, targets, ground_truth = [], [], []
    for samples, batch_targets in loader:
        output = model(samples.to(device))
        for i, target in enumerate(batch_targets):
            predictions.append(postprocess_variant(output["pred_logits"][i].cpu(),
                                                   output["pred_boxes"][i].cpu(),
                                                   target["orig_size"], "perclass"))
            targets.append(target)
            height, width = target["orig_size"].tolist()
            boxes = box_cxcywh_to_xyxy(target["boxes"]) * torch.tensor([width, height, width, h
    eight])
            gt = {}
            for box, label in zip(boxes.numpy(), target["labels"].tolist()):
                gt.setdefault(label, []).append(box)
            ground_truth.append(gt)
    model.train(was_training)
    legacy = {c: float(ap_from_preds(predictions, ground_truth, c)[0])
              for c in range(len(NEU_DET_CLASSES))}
    standard = coco_ap50(predictions, targets, NEU_DET_CLASSES)
    def average(scores, ids):
        values = [scores[c] for c in ids if scores[c] is not None]
        return float(np.mean(values)) if values else 0.
    old, new = average(standard, old_classes), average(standard, new_classes)
    return {"metric_protocol": "COCO_AP50_perclassNMS_top100",
            "old_ap": old, "new_ap": new, "overall_ap": (old + new) / 2,
            "legacy_old_ap": average(legacy, old_classes),
            "legacy_new_ap": average(legacy, new_classes),
            "per_class_ap50": standard, "per_class_legacy_ap": legacy}
def _assert_frozen(model, frozen):
```

```
    current = model.state_dict()
    changed = [name for name, initial in frozen.items()
               if not torch.equal(initial, current[name].detach().cpu())]
    if changed:
        raise RuntimeError(f"Frozen old branch changed: {changed[:5]}")
def train_dual_phase(trainer, previous, old_classes, new_classes, phase_id, sample_count, is_fi
    netune=False):
    # Keep the public trainer interface for UI/scripts and the existing epoch runner.
    from incremental_train import ReplayDataset, unwrap_model
    args = trainer.args
    emit = getattr(trainer, "workflow_event", lambda *a, **kw: None)
    emit("分支初始化")
    if args.epochs_incremental < 1 or min(args.dual_lr, args.dual_head_lr) <= 0:
        raise ValueError("dual requires at least one epoch and positive learning rates")
    if is_finetune or hasattr(unwrap_model(previous), "new_branch"):
        raise ValueError("dual starts from a compact phase-0 checkpoint; use a fresh output dir
    ectory")
    if args.warmstart_epochs != 0:
        raise ValueError("dual trains all new decoder parameters from epoch 0; set --warmstart_
    epochs 0")
    old_classes, new_classes = sorted(old_classes), sorted(new_classes)
    current_classes = sorted(old_classes + new_classes)
    if current_classes != list(range(len(NEU_DET_CLASSES))):
        raise ValueError("dual currently supports one old/new split covering all NEU-DET classe
    s")
    args.dual_old_class_ids, args.dual_new_class_ids = old_classes, new_classes
    previous = unwrap_model(previous)
    args.num_queries = previous.num_queries
    expanded = copy.deepcopy(previous)
    expanded.teacher_model = None
    expanded.class_global_ids = list(old_classes)
    expanded.expand_classification_head(len(current_classes), old_class_global_ids=old_classes)
    model, criterion = build_incremental_model(args, num_classes=len(current_classes))
    state = expanded.state_dict()
    # This is the only permitted initialization gap, never used for inference loads.
    state.update({k: v for k, v in model.state_dict().items() if k.startswith("new_branch.")})
    model.load_state_dict(state, strict=True)
    model.new_branch.initialize_from_old(model)
    model.freeze_old_branch()
    del expanded
    frozen = {name: value.detach().cpu().clone() for name, value in model.state_dict().items()
              if not name.startswith("new_branch.")}
    trainer._bbox_anchor = None
    trainer._bbox_norm0 = None
    trainer._ema_state = None
    groups = {"decoder": [], "heads": []}
    for name, parameter in model.new_branch.named_parameters():
        groups["heads" if name.startswith(("class_embed.", "bbox_embed.")) else "decoder"].appe
    nd(parameter)
    optimizer = torch.optim.AdamW([
```

```
    {"params": groups["decoder"], "lr": args.dual_lr},
    {"params": groups["heads"], "lr": args.dual_head_lr},
], weight_decay=args.weight_decay)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args.epochs_increme
ntal,
                                                       eta_min=min(args.dual_lr, args.dual_
head_lr) * .05)
dataset = getattr(trainer, "workflow_new_dataset", None)
if dataset is None:
    dataset = build_small_sample_dataset(args, phase_classes=current_classes,
                                         new_class_global_ids=new_classes,
                                         samples_per_class=sample_count, image_set="train")
if not len(dataset):
    raise RuntimeError("Empty new-class training subset")
replay_dataset = getattr(trainer, "workflow_replay_dataset", None)
if replay_dataset is not None:
    replay = replay_dataset
    training = ConcatDataset([dataset, replay_dataset]) if len(replay_dataset) else dataset
else:
    replay = list(trainer.replay_buffer.get_samples(current_global_classes=current_classes)
)[:len(dataset)]
    training = ConcatDataset([dataset, ReplayDataset(replay)]) if replay else dataset
loader = DataLoader(training, batch_size=args.batch_size, shuffle=True, drop_last=False,
                    num_workers=args.num_workers, collate_fn=misc.collate_fn)
validation = getattr(trainer, "workflow_validation_dataset", None)
if validation is None:
    validation = build_neu_det_dataset("val", args, phase_classes=current_classes)
val_loader = DataLoader(validation, batch_size=4, shuffle=False, num_workers=0,
                        collate_fn=misc.collate_fn)
print(f"[Dual] Shared encoder frozen; old decoder frozen; new decoder fully trainable "
      f"({sum(p.numel() for p in model.new_branch.parameters()):,} parameters)")
print(f"[Dual] Queries old={model.num_queries} new={model.new_branch.query_embed.num_embedd
ings}; "
      f"4D iterative refinement; decoder LR={args.dual_lr:g}, head LR={args.dual_head_lr:g}
")
print(f"[Data] Fixed {len(dataset)} new images ({sample_count}/class), replay={len(replay)}
; "
      "fresh augmentation each epoch")
classification = "quality focal (detached IoU targets)" if criterion.quality_loss else "foc
al"
print(f"[Dual] New branch: {classification} + L1 + GIoU and auxiliary layers; old replay su
pplies negatives.")
if model.requires_training_targets:
    print(f"[Dual] Training-only DN groups={model.new_branch.denoising.groups}; "
          "normal queries are masked from DN keys; validation/inference receives images onl
y.")
print("[Metric] Checkpoints selected by COCO AP50. Legacy AP is also printed for historical
 comparison.")
emit("扩展模型初始评估")
baseline = evaluate_dual(model, val_loader, trainer.device, old_classes, new_classes)
```

```
emit("扩展模型初始评估完成", metrics=baseline)
print(f"[BeforeTrain] COCO old={baseline['old_ap']:.4f} new={baseline['new_ap']:.4f}; "
      f"legacy old={baseline['legacy_old_ap']:.4f} new={baseline['legacy_new_ap']:.4f}")
trainer._append_log({"epoch": -1, "phase": phase_id, **baseline})
best_new, best_overall, best_legacy = -1., -1., -1.
old_history = []
start = time.time()
for epoch in range(args.epochs_incremental):
    emit("增量训练", epoch=epoch + 1, epochs=args.epochs_incremental, batch=0, batches=len(
loader))
    stats = trainer.train_one_epoch(model, criterion, loader, optimizer, trainer.device, ep
och)
    scheduler.step()
    emit("逐轮评估", epoch=epoch + 1, epochs=args.epochs_incremental)
    metrics = evaluate_dual(model, val_loader, trainer.device, old_classes, new_classes)
    _assert_frozen(model, frozen)
    old_history.append(metrics["old_ap"])
    print(f"[Eval COCO AP50] old={metrics['old_ap']:.4f} new={metrics['new_ap']:.4f} "
          f"overall={metrics['overall_ap']:.4f}")
    print(f"[Eval Legacy AP] old={metrics['legacy_old_ap']:.4f} new={metrics['legacy_new_ap
']:.4f}")
    print("[PerClass COCO AP50] " + " | ".join(
        f"{NEU_DET_CLASSES[c]}={value:.4f}" if value is not None else f"{NEU_DET_CLASSES[c]
}=N/A"
        for c, value in metrics["per_class_ap50"].items()))
    print(f"[FrozenAudit] {len(frozen)} old/shared tensors unchanged")
    trainer._append_log({"epoch": epoch, "phase": phase_id, **metrics,
                         **{f"train_{key}": value for key, value in stats.items()}})
    if metrics["new_ap"] > best_new:
        best_new = metrics["new_ap"]
        trainer._save_checkpoint(model, optimizer, scheduler, epoch, phase_id, "best_new",
metrics)
    if metrics["overall_ap"] > best_overall:
        best_overall = metrics["overall_ap"]
        trainer._save_checkpoint(model, optimizer, scheduler, epoch, phase_id, "best_overal
l", metrics)
    legacy_overall = (metrics["legacy_old_ap"] + metrics["legacy_new_ap"]) / 2
    if legacy_overall > best_legacy:
        best_legacy = legacy_overall
        trainer._save_checkpoint(model, optimizer, scheduler, epoch, phase_id, "best_legacy
", metrics)
    trainer._save_checkpoint(model, optimizer, scheduler, epoch, phase_id, metrics=metrics)
    emit("本轮已保存", epoch=epoch + 1, epochs=args.epochs_incremental,
         metrics=metrics, frozen_tensors=len(frozen), losses=stats)
final_path = trainer._save_checkpoint(model, optimizer, scheduler, epoch, phase_id, "model"
, metrics)
summary = {"phase": phase_id, "architecture": "dual", "best_new_ap": best_new,
           "best_overall_ap": best_overall, "best_legacy_ap": best_legacy,
           "old_class_acc_history": old_history, "metric_protocol": metrics["metric_protoco
l"]}
```

```
    trainer.history["phase_metrics"].append(summary)
    (trainer.output_dir / "dual_summary.json").write_text(json.dumps(summary, indent=2), encodi
    ng="utf-8")
    print(f"\n[Phase {phase_id} Complete] Time: {(time.time()-start)/60:.2f} min; "
          f"best new COCO AP50={best_new:.4f}; best overall={best_overall:.4f}")
    print(f"[Checkpoint] Final model: {final_path}")
    return model, current_classes
# 文件 models/dual_branch_detr.py
"""Independent incremental decoder with four-coordinate box refinement.
The inherited detector owns the frozen old path. Only ``new_branch`` learns;
classification columns are disjoint when predictions are fused.
"""
import copy
import math
import torch
from torch import nn
from torch.nn import functional as F
from models.deformable_detr import MLP
from models.dual_denoising import DenoisingQueries, branch_targets
from models.incremental_detr import IncrementalDeformableDETR
from util import box_ops
from util.misc import inverse_sigmoid
class NewDetectionBranch(nn.Module):
    def __init__(self, detector, num_classes, num_queries, dn_groups=0,
                 dn_box_noise=.4, dn_label_noise=.2):
        super().__init__()
        dim = detector.transformer.d_model
        self.layers = copy.deepcopy(detector.transformer.decoder.layers)
        self.query_embed = nn.Embedding(num_queries, dim * 2)
        self.reference_points = nn.Linear(dim, 4)
        self.class_embed = nn.ModuleList([nn.Linear(dim, num_classes) for _ in self.layers])
        self.bbox_embed = nn.ModuleList([MLP(dim, dim, 4, 3) for _ in self.layers])
        if dn_groups < 0:
            raise ValueError("dn_groups cannot be negative")
        self.denoising = (DenoisingQueries(dim, num_classes, dn_groups, dn_box_noise, dn_label_
    noise)
                          if dn_groups else None)
        self.initialize_from_old(detector)
    @torch.no_grad()
    def initialize_from_old(self, detector):
        self.layers.load_state_dict(detector.transformer.decoder.layers.state_dict(), strict=Tr
    ue)
        indices = torch.arange(self.query_embed.num_embeddings, device=detector.query_embed.wei
    ght.device)
        indices = indices % detector.query_embed.num_embeddings
        self.query_embed.weight.copy_(detector.query_embed.weight[indices])
        self.reference_points.weight.zero_()
        self.reference_points.bias.fill_(math.log(.2 / .8))
        self.reference_points.weight[:2].copy_(detector.transformer.reference_points.weight)
        self.reference_points.bias[:2].copy_(detector.transformer.reference_points.bias)
```

```
        for layer, cls, box in zip(self.layers, self.class_embed, self.bbox_embed):
            # A base trained with adapters is outside this architecture's contract.
            if getattr(layer, "adapter", None) is not None:
                raise ValueError("dual requires a baseline base checkpoint without adapters")
            nn.init.normal_(cls.weight, std=.01)
            nn.init.constant_(cls.bias, math.log(.01 / .99))
            # Transfer useful hidden transforms; geometry starts from explicit 4D refs.
            box.load_state_dict(detector.bbox_embed[0].state_dict())
            nn.init.zeros_(box.layers[-1].weight)
            nn.init.zeros_(box.layers[-1].bias)
    def forward(self, encoded, targets=None):
        memory, mask, shapes, starts, ratios = encoded
        position, hidden = self.query_embed.weight.chunk(2, dim=-1)
        position = position.unsqueeze(0).expand(memory.shape[0], -1, -1)
        hidden = hidden.unsqueeze(0).expand(memory.shape[0], -1, -1)
        reference = self.reference_points(position).sigmoid()
        dn = self.denoising(targets, hidden.shape[1]) if self.training and self.denoising and t
    argets is not None else None
        dn_count = dn["count"] if dn else 0
        if dn:
            hidden = torch.cat([dn["hidden"], hidden], dim=1)
            position = torch.cat([dn["position"], position], dim=1)
            reference = torch.cat([dn["reference"], reference], dim=1)
        predictions = []
        for layer, cls, box in zip(self.layers, self.class_embed, self.bbox_embed):
            scaled_reference = reference[:, :, None] * torch.cat([ratios, ratios], -1)[:, None]
            hidden = layer(hidden, position, scaled_reference, memory, shapes, starts, mask,
                           self_attn_mask=dn["attention_mask"] if dn else None,
                           query_padding_mask=dn["padding_mask"] if dn else None)
            coordinates = (box(hidden) + inverse_sigmoid(reference)).sigmoid()
            predictions.append({"pred_logits": cls(hidden), "pred_boxes": coordinates})
            # Each layer has direct supervision; the next layer refines its four coords.
            reference = coordinates.detach()
        normal = [{key: value[:, dn_count:] for key, value in output.items()} for output in pre
    dictions]
        result = {**normal[-1], "aux_outputs": normal[:-1]}
        if dn:
            denoised = [{key: value[:, :dn_count] for key, value in output.items()} for output
    in predictions]
            result.update(dn_outputs={**denoised[-1], "aux_outputs": denoised[:-1]},
                          dn_indices=dn["indices"], dn_valid=dn["valid"])
        return result
class DualBranchDeformableDETR(IncrementalDeformableDETR):
    def __init__(self, *args, old_class_ids, new_class_ids, new_num_queries=None,
                 dn_groups=0, dn_box_noise=.4, dn_label_noise=.2, **kwargs):
        super().__init__(*args, **kwargs)
        if self.two_stage or self.head_adapters is not None:
            raise ValueError("dual supports single-stage baseline base checkpoints")
        self.old_class_ids = list(old_class_ids)
        self.new_class_ids = list(new_class_ids)
```

```
        self.class_global_ids = sorted(self.old_class_ids + self.new_class_ids)
        if (set(self.old_class_ids) & set(self.new_class_ids)
                or len(self.class_global_ids) != self.current_num_classes
                or len(set(self.class_global_ids)) != self.current_num_classes):
            raise ValueError("Old/new classes must be a disjoint partition of model columns")
        if new_num_queries is not None and new_num_queries < 1:
            raise ValueError("new_num_queries must be positive")
        self.new_branch = NewDetectionBranch(self, len(new_class_ids), new_num_queries or self.
    num_queries,
                                              dn_groups, dn_box_noise, dn_label_noise)
        self.requires_training_targets = bool(dn_groups)
        self.freeze_old_branch()
    def freeze_old_branch(self):
        for name, parameter in self.named_parameters():
            parameter.requires_grad_(name.startswith("new_branch."))
        self.train(self.training)
    def train(self, mode=True):
        # Frozen weights alone do not disable dropout or mutable backbone statistics.
        super().train(False)
        self.training = mode
        if hasattr(self, "new_branch"):
            self.new_branch.train(mode)
        return self
    def forward_from_features(self, srcs, masks, pos, encoded=None, targets=None):
        if targets is not None and not self.training:
            raise ValueError("Evaluation/inference must receive images only, without targets")
        if self.training and self.requires_training_targets and targets is None:
            raise ValueError("DN training requires targets; use model(samples, targets=targets)
    ")
        with torch.no_grad():
            if encoded is None:
                encoded = self.transformer.encode(srcs, masks, pos)
            old = super().forward_from_features(srcs, masks, pos, encoded=encoded)
        local_targets = branch_targets(targets, self.new_class_ids, self.class_global_ids) if t
    argets is not None else None
        new = self.new_branch(encoded, local_targets)
        if not self.aux_loss:
            new.pop("aux_outputs", None)
            if "dn_outputs" in new:
                new["dn_outputs"].pop("aux_outputs", None)
        old_columns = [self.class_global_ids.index(c) for c in self.old_class_ids]
        new_columns = [self.class_global_ids.index(c) for c in self.new_class_ids]
        old_logits = torch.full_like(old["pred_logits"], -1e4)
        old_logits[..., old_columns] = old["pred_logits"][..., old_columns]
        new_logits = new["pred_logits"].new_full(
            (*new["pred_logits"].shape[:2], self.current_num_classes), -1e4)
        new_logits[..., new_columns] = new["pred_logits"]
        return {
            "pred_logits": torch.cat([old_logits, new_logits], dim=1),
            "pred_boxes": torch.cat([old["pred_boxes"], new["pred_boxes"]], dim=1),
```

```
            "old_outputs": old,
            "new_outputs": new,
        }
class NewBranchCriterion(nn.Module):
    """Match human new-class GT only. Old replay images supply negative examples."""
    def __init__(self, matcher, new_ids, global_ids, layers, alpha=.25,
                 cls_weight=2., bbox_weight=5., giou_weight=2., quality_loss=False, dn_weight=1
    .):
        super().__init__()
        self.matcher = matcher
        self.alpha = alpha
        self.new_ids = list(new_ids)
        self.global_ids = list(global_ids)
        self.quality_loss = quality_loss
        if dn_weight < 0:
            raise ValueError("dn_weight cannot be negative")
        self.weight_dict = {"loss_ce": cls_weight, "loss_bbox": bbox_weight, "loss_giou": giou_
    weight}
        self.weight_dict.update({f"{key}_{i}": value
                                  for i in range(layers - 1)
                                  for key, value in list(self.weight_dict.items())})
        self.weight_dict.update({key.replace("loss_", "loss_dn_", 1): value * dn_weight
                                  for key, value in list(self.weight_dict.items())})
    def branch_targets(self, targets):
        return branch_targets(targets, self.new_ids, self.global_ids)
    def layer_losses(self, output, targets, log=False, indices=None, valid=None, quality=None):
        if indices is None:
            indices = self.matcher(output, targets)
        use_quality = self.quality_loss if quality is None else quality
        logits, boxes = output["pred_logits"], output["pred_boxes"]
        onehot = torch.zeros_like(logits)
        source, expected = [], []
        correct, matched = logits.new_zeros(()), 0
        for batch_id, (prediction, target) in enumerate(indices):
            scores = 1.
            if use_quality and len(prediction):
                scores = box_ops.box_iou(box_ops.box_cxcywh_to_xyxy(boxes[batch_id, prediction]
    .detach()),
                                         box_ops.box_cxcywh_to_xyxy(targets[batch_id]["boxes"][
    target]))[0].diag().clamp(0, 1)
            onehot[batch_id, prediction, targets[batch_id]["labels"][target]] = scores
            source.append(boxes[batch_id, prediction])
            expected.append(targets[batch_id]["boxes"][target])
            if log:
                correct += (logits[batch_id, prediction].argmax(-1)
                            == targets[batch_id]["labels"][target]).float().sum()
                matched += len(target)
        normalizer = max(sum(len(prediction) for prediction, _ in indices) if valid is not None
                         else sum(len(t["labels"]) for t in targets), 1)
        probabilities = logits.sigmoid()
```

```
        ce = F.binary_cross_entropy_with_logits(logits, onehot, reduction="none")
        if use_quality:
            # Quality focal loss: matched labels target their detached IoU,
            # all unmatched/class-negative entries target zero. Scores retain
            # the same meaning in training and inference; no post-hoc multiplier.
            classification = ce * (onehot - probabilities).abs().pow(2)
        else:
            p_t = probabilities * onehot + (1 - probabilities) * (1 - onehot)
            alpha_t = self.alpha * onehot + (1 - self.alpha) * (1 - onehot)
            classification = ce * (1 - p_t).pow(2) * alpha_t
        if valid is not None:
            classification = classification * valid[..., None]
        focal = classification.sum() / normalizer
        source, expected = torch.cat(source), torch.cat(expected)
        l1 = F.l1_loss(source, expected, reduction="sum") / normalizer
        giou = box_ops.generalized_box_iou(box_ops.box_cxcywh_to_xyxy(source),
                                           box_ops.box_cxcywh_to_xyxy(expected))
        giou_loss = (1 - torch.diag(giou)).sum() / normalizer
        losses = {"loss_ce": focal, "loss_bbox": l1, "loss_giou": giou_loss}
        if log:
            losses["class_error"] = 100 * (1 - correct / matched) if matched else correct
        return losses
    def forward(self, outputs, targets, **unused):
        targets = self.branch_targets(targets)
        new = outputs["new_outputs"]
        losses = self.layer_losses(new, targets, log=True)
        for i, auxiliary in enumerate(new.get("aux_outputs", [])):
            losses.update({f"{key}_{i}": value
                           for key, value in self.layer_losses(auxiliary, targets).items()})
        if "dn_outputs" in new:
            dn_layers = [("", new["dn_outputs"])] + [(f"_{i}", output)
                         for i, output in enumerate(new["dn_outputs"].get("aux_outputs", []))]
            for suffix, output in dn_layers:
                # Known clean assignments, never Hungarian or quality targets.
                values = self.layer_losses(output, targets, indices=new["dn_indices"],
                                           valid=new["dn_valid"], quality=False)
                losses.update({key.replace("loss_", "loss_dn_", 1)+suffix: value for key, value
     in values.items()})
        return losses
# 文件 models/dual_denoising.py
"""Training-only known assignments for the independent new detector branch."""
import torch
from torch import nn
from models.deformable_detr import MLP
from util.misc import inverse_sigmoid
def branch_targets(targets, new_ids, global_ids):
    result = []
    for target in targets:
        labels = target["labels"]
        local = torch.full_like(labels, -1)
```

```
        for branch_id, global_id in enumerate(new_ids):
            local[labels == global_ids.index(global_id)] = branch_id
        keep = local >= 0
        result.append({"labels": local[keep], "boxes": target["boxes"][keep]})
    return result
class DenoisingQueries(nn.Module):
    def __init__(self, dim, classes, groups, box_noise=.4, label_noise=.2):
        super().__init__()
        if groups < 1 or not 0 <= box_noise <= 1 or not 0 <= label_noise <= 1:
            raise ValueError("DN requires positive groups and noise scales within [0, 1]")
        self.groups, self.box_noise, self.label_noise = groups, box_noise, label_noise
        self.classes = classes
        self.label_embed = nn.Embedding(classes + 1, dim)
        self.box_position = MLP(4, dim, dim, 2)
    def forward(self, targets, normal_queries):
        maximum = max((len(t["labels"]) for t in targets), default=0)
        if maximum == 0:
            return None  # Old-only/empty batches have ordinary negative supervision.
        count = maximum * self.groups
        device = self.label_embed.weight.device
        boxes = self.label_embed.weight.new_full((len(targets), count, 4), .5)
        labels = torch.full((len(targets), count), self.classes, dtype=torch.long, device=devic
    e)
        valid = torch.zeros_like(labels, dtype=torch.bool)
        indices = []
        for batch, target in enumerate(targets):
            length = len(target["labels"])
            slots = (torch.arange(self.groups, device=device)[:, None] * maximum
                     + torch.arange(length, device=device)[None]).flatten()
            expected = torch.arange(length, device=device).repeat(self.groups)
            indices.append((slots, expected))
            valid[batch, slots] = True
            labels[batch, slots] = target["labels"][expected]
            clean = target["boxes"][expected]
            delta_scale = torch.cat([clean[:, 2:] * .5, clean[:, 2:]], dim=-1)
            boxes[batch, slots] = (clean + (torch.rand_like(clean)*2-1)
                                   * delta_scale * self.box_noise).clamp(1e-4, 1-1e-4)
        corrupt = (torch.rand(labels.shape, device=device) < self.label_noise) & valid
        labels = torch.where(corrupt, torch.randint(self.classes, labels.shape, device=device),
     labels)
        # True means a blocked key. Normal queries cannot access any GT-derived
        # DN key at any layer. Separate DN groups cannot access one another.
        attention_mask = torch.zeros((count+normal_queries, count+normal_queries),
                                     dtype=torch.bool, device=device)
        attention_mask[count:, :count] = True
        for group in range(self.groups):
            start, end = group*maximum, (group+1)*maximum
            attention_mask[start:end, :start] = True
            attention_mask[start:end, end:count] = True
        padding_mask = torch.cat([~valid, torch.zeros((len(targets), normal_queries),
```

```
                                                dtype=torch.bool, device=device)], dim=1)
        return {"hidden": self.label_embed(labels), "position": self.box_position(inverse_sigmo
    id(boxes)),
                "reference": boxes, "attention_mask": attention_mask,
                "padding_mask": padding_mask, "valid": valid, "indices": indices, "count": coun
    t}
# 文件 datasets/neu_det.py
"""
NEU-DET Dataset Loader for Deformable DETR Incremental Learning
将 YOLO 格式标签转换为 Deformable DETR 所需的格式
支持增量学习阶段的类别子集选择
"""
from pathlib import Path
import torch
import torch.utils.data
import os
from PIL import Image
import random
import numpy as np
from util.misc import get_local_rank, get_local_size
import datasets.transforms as T
# NEU-DET 6 类缺陷定义
NEU_DET_CLASSES = ['crazing', 'inclusion', 'patches', 'pitted_surface', 'rolled-in_scale', 'scr
    atches']
# ============================================================
# 多组类别划分方案 (消除难易度偏差)
# 每组: 3 个旧类(全量训练) + 3 个新类(小样本增量)
# 通过旋转划分，每个类别都会经历"旧类"和"新类"
#
# Split 1: 旧=裂纹/夹杂/麻点(crazing/inclusion/pitted_surface)
#          新=斑块/划痕/氧化铁皮(patches/scratches/rolled-in_scale)
# Split 2: 旧=裂纹/斑块/氧化铁皮  新=夹杂/麻点/划痕
# Split 3: 旧=夹杂/划痕/麻点      新=裂纹/斑块/氧化铁皮
# ============================================================
SPLIT_GROUPS = {
    # Split 1: 大面积+中等 vs 小面积+细长
    '1': {'old': [0, 1, 3], 'new': [2, 4, 5]},   # old=crazing,inclusion,pitted | new=patches,s
    cale,scratches
    # Split 2: 混合难度旋转
    '2': {'old': [0, 2, 4], 'new': [1, 3, 5]},   # old=crazing,patches,scale | new=inclusion,pi
    tted,scratches
    # Split 3: 另一种混合
    '3': {'old': [1, 3, 5], 'new': [0, 2, 4]},   # old=inclusion,pitted,scratches | new=crazing
    ,patches,scale
}
# 新类小样本量配置（模拟工业现场样本不足；60/100 供样本量敏感性分析）
NEW_CLASS_SAMPLE_SIZES = [5, 10, 30, 60, 100]
# 兼容旧接口
INCREMENTAL_PHASES = {
    0: SPLIT_GROUPS['1']['old'],
```

```
    1: SPLIT_GROUPS['1']['old'] + SPLIT_GROUPS['1']['new'],
}
class NEUDETDataset(torch.utils.data.Dataset):
    """NEU-DET 数据集，支持增量学习的类别子集"""
    def __init__(self, img_folder, label_folder, phase_classes, transforms=None,
                 return_masks=False, cache_mode=False, local_rank=0, local_size=1):
        """
        Args:
            img_folder: 图片文件夹路径
            label_folder: 标签文件夹路径 (YOLO 格式 txt)
            phase_classes: 当前阶段使用的类别 ID 列表 (如 [0, 5] 表示只使用 crazing 和 scratches)
            transforms: 数据增强变换
            return_masks: 是否返回 mask
            cache_mode: 是否缓存到内存
            local_rank: 分布式训练 rank
            local_size: 分布式训练 size
        """
        self.img_folder = Path(img_folder)
        self.label_folder = Path(label_folder)
        self.phase_classes = phase_classes
        self._transforms = transforms
        self.return_masks = return_masks
        # 构建类别映射: 全局类别 ID -> 当前阶段局部 ID
        # 未在当前阶段的类别映射为-1（忽略）
        self.global_to_local = {gid: lid for lid, gid in enumerate(phase_classes)}
        self.num_classes = len(phase_classes)
        # 加载所有图片文件
        self.img_files = sorted([
            f for f in self.img_folder.iterdir()
            if f.suffix.lower() in ['.jpg', '.jpeg', '.png', '.bmp']
        ])
        print(f"[NEU-DET] Loaded {len(self.img_files)} images "
              f"for classes {[NEU_DET_CLASSES[c] for c in phase_classes]}")
    def __len__(self):
        return len(self.img_files)
    def get_global_class_ids(self, idx):
        """Read class membership without image decoding or random augmentation."""
        label_path = self.label_folder / (self.img_files[idx].stem + '.txt')
        if not label_path.exists():
            return set()
        classes = set()
        for line in label_path.read_text(encoding='utf-8').splitlines():
            parts = line.split()
            if len(parts) >= 5:
                class_id = int(parts[0])
                if class_id in self.global_to_local:
                    classes.add(class_id)
        return classes
    def __getitem__(self, idx):
        img_path = self.img_files[idx]
```

```
        img = Image.open(img_path).convert('RGB')
        w, h = img.size
        # 加载对应的 YOLO 格式标签
        label_path = self.label_folder / (img_path.stem + '.txt')
        boxes = []
        classes = []
        if label_path.exists():
            with open(label_path, 'r') as f:
                for line in f.readlines():
                    parts = line.strip().split()
                    if len(parts) >= 5:
                        cls_id = int(parts[0])
                        # 只保留当前阶段关注的类别
                        if cls_id in self.phase_classes:
                            cx, cy, bw, bh = float(parts[1]), float(parts[2]), float(parts[3]),
    float(parts[4])
                            # YOLO 格式转 xyxy 格式（归一化坐标）
                            x1 = (cx - bw / 2) * w
                            y1 = (cy - bh / 2) * h
                            x2 = (cx + bw / 2) * w
                            y2 = (cy + bh / 2) * h
                            boxes.append([x1, y1, x2, y2])
                            # 转换为局部类别 ID
                            classes.append(self.global_to_local[cls_id])
        if len(boxes) == 0:
            boxes = torch.zeros((0, 4), dtype=torch.float32)
            classes = torch.zeros((0,), dtype=torch.int64)
            area = torch.zeros((0,), dtype=torch.float32)
        else:
            boxes = torch.as_tensor(boxes, dtype=torch.float32)
            classes = torch.as_tensor(classes, dtype=torch.int64)
            area = (boxes[:, 2] - boxes[:, 0]) * (boxes[:, 3] - boxes[:, 1])
        target = {
            'boxes': boxes,
            'labels': classes,
            'image_id': torch.tensor([idx]),
            'area': area,
            'iscrowd': torch.zeros((len(classes),), dtype=torch.int64),
            'orig_size': torch.as_tensor([int(h), int(w)]),
            'size': torch.as_tensor([int(h), int(w)]),
        }
        if self._transforms is not None:
            img, target = self._transforms(img, target)
        return img, target
class ExperienceReplayBuffer:
    """经验回放缓冲区
    Fix: 统一标签体系 — 存储时转为全局 ID，取出时按当前阶段映射为局部 ID。
         解决回放/数据集/评估之间的标签错位问题。
    """
    def __init__(self, max_per_class=50):
```

```
        self.max_per_class = max_per_class
        # key = 全局类别 ID (NEU-DET 0~5)
        self.buffer = {}  # {global_cls_id: [(img_tensor, target_with_global_labels), ...]}
    def add_sample(self, img, target, local_to_global_map):
        """添加样本，将局部标签转为全局标签存储
        Args:
            img: 图像 tensor
            target: target dict（labels 使用当前阶段的局部 ID）
            local_to_global_map: {local_id: global_id} 映射表
        """
        if target['labels'].numel() == 0:
            return
        # 转换为全局标签存储
        global_labels = [local_to_global_map.get(l.item(), l.item())
                         for l in target['labels']]
        global_target = {k: (v.clone() if isinstance(v, torch.Tensor) else v)
                         for k, v in target.items()}
        global_target['labels'] = torch.tensor(global_labels, dtype=target['labels'].dtype)
        # 按全局类 ID 分组存储
        for gcls in set(global_labels):
            if gcls not in self.buffer:
                self.buffer[gcls] = []
            if len(self.buffer[gcls]) < self.max_per_class:
                self.buffer[gcls].append((img.clone(), global_target))
    def get_samples(self, current_global_classes, n_samples=None, class_priority=None):
        """
        取出样本，将全局标签映射为当前阶段的局部 ID
        Args:
            current_global_classes: 当前阶段的全局类别 ID 列表，如 [0,1,3,2,4,5]
                                    局部 ID = 列表中的索引
            n_samples: 最大返回数
            class_priority: {全局类别 ID: 采样权重} — AP 越低的类权重越高（创新点 1：低 AP 优先回放
    )
        """
        all_samples = []
        weights = []
        class_priority = class_priority or {}
        for gcls_id, samples in self.buffer.items():
            # 只返回属于当前阶段类别范围的样本
            if gcls_id not in current_global_classes:
                continue
            # 创新点 1：类别优先级（AP 越低权重越高，默认 1.0，最高 4x）
            priority = float(class_priority.get(gcls_id, 1.0))
            for img, target in samples:
                all_samples.append((img, target))
                n_boxes = len(target.get('labels', []))
                weights.append(max(n_boxes, 1) * priority)
        if not all_samples:
            return []
        if n_samples and n_samples < len(all_samples):
```

```
            total_w = sum(weights)
            probs = [w / total_w for w in weights]
            indices = np.random.choice(len(all_samples), size=n_samples,
                                       replace=False, p=probs)
            all_samples = [all_samples[i] for i in indices]
        # 全局 ID → 当前阶段局部 ID
        global_to_local = {g: i for i, g in enumerate(current_global_classes)}
        result = []
        for img, target in all_samples:
            new_target = {k: (v.clone() if isinstance(v, torch.Tensor) else v)
                          for k, v in target.items()}
            if 'labels' in new_target:
                local_labels = [global_to_local.get(l.item(), 0)
                                for l in new_target['labels']]
                new_target['labels'] = torch.tensor(local_labels,
                                                    dtype=target['labels'].dtype)
            result.append((img, new_target))
        return result
    def __len__(self):
        return sum(len(v) for v in self.buffer.values())
class _RotateIfNewClass:
    """v107：只对含新类实例的样本做 0/90/180/270 等概随机旋转。
    旧类样本不旋转——基座 Phase 0 训练时没见过旋转输入，冻结的
    backbone/encoder 对旋转后的旧类图是 OOD，会白送旧类 AP。
    混合图（新旧类同框）整图旋转：缺陷语义随图一起变，框坐标由
    T.rot90 同步变换。与 RandomHorizontalFlip 组合后方向覆盖 8 种，
    专治 scratches"只见过竖直划痕"导致的方向先验塌缩。
    """
    def __init__(self, new_local_ids):
        self.new_ids = set(new_local_ids or [])
    def __call__(self, img, target):
        labels = target.get("labels", None)
        if labels is not None and len(labels) > 0:
            if any(l in self.new_ids for l in labels.tolist()):
                return T.rot90(img, target, random.randint(0, 3))
        return img, target
def make_neu_det_transforms(image_set, rot_new_ids=None, val_size=None):
    """
    NEU-DET 专用数据增强
    NEU-DET 图片均为 200x200 小图，不需要缩放到 800+像素
    rot_new_ids: 触发 90 度随机旋转的新类局部 id 列表（仅 train 生效）
    """
    # 灰度图三通道同值归一化（NEU-DET 原图为灰度，convert('RGB')后 R=G=B）
    normalize = T.Compose([
        T.ToTensor(),
        T.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
    ])
    # NEU-DET 原图 200x200，配合 FPN 5 层(stride=4)使用
    # stride=4 层: 8px 划痕 @ 200 输入 = 2 像素 ← 可被 deformable attention 采样!
    scales = [200, 224, 256, 288, 320]
```

```
    if image_set == 'train':
        tf = []
        if rot_new_ids:
            tf.append(_RotateIfNewClass(rot_new_ids))
        tf += [
            T.RandomHorizontalFlip(p=0.3),
            T.RandomResize(scales, max_size=480),
            normalize,
        ]
        return T.Compose(tf)
    if image_set == 'val':
        # NEU_VAL_SIZE 环境变量可覆盖推理输入尺寸（默认 224）。
        # 400px 已验证可提升细长新类的框紧致度（亚 cell 精度瓶颈），
        # 仅影响 eval/诊断脚本，训练仍走 200~320 多尺度。
        _val_size = int(val_size if val_size is not None else os.environ.get('NEU_VAL_SIZE', 22
    4))
        return T.Compose([
            T.RandomResize([_val_size], max_size=max(480, _val_size)),
            normalize,
        ])
    raise ValueError(f'unknown {image_set}')
def build_neu_det_dataset(image_set, args, phase_classes=None):
    """构建 NEU-DET 数据集"""
    root = Path(args.neu_det_path)
    assert root.exists(), f'NEU-DET path {root} does not exist'
    if phase_classes is None:
        phase_classes = list(range(6))
    # NEU-DET 使用'valid'目录而非'val'
    folder_name = 'valid' if image_set == 'val' else image_set
    img_folder = root / folder_name / 'images'
    label_folder = root / folder_name / 'labels'
    # v107：新类 90 度旋转增强。target 里的 labels 是局部 id（global_to_local
    # 映射后），因此把全局新类 id 映射为当前阶段的局部 id；phase 0 时新类不在
    # phase_classes 中 -> rot_ids=[] -> 自动关闭。
    rot_ids = None
    if image_set == 'train':
        split = SPLIT_GROUPS[str(getattr(args, 'split_group', '1'))]
        rot_ids = [phase_classes.index(g) for g in split['new']
                   if g in phase_classes]
        if rot_ids:
            print(f"[NEU-DET] 新类 90 度旋转增强已启用，局部 id={rot_ids}")
    dataset = NEUDETDataset(
        img_folder=img_folder,
        label_folder=label_folder,
        phase_classes=phase_classes,
        transforms=make_neu_det_transforms(image_set, rot_new_ids=rot_ids),
        return_masks=args.masks,
        cache_mode=args.cache_mode,
        local_rank=get_local_rank(),
        local_size=get_local_size()
```

```
    )
    return dataset
class SmallSampleNEUDETDataset(torch.utils.data.Dataset):
    """
    小样本数据集：从全量数据集中按类别采样指定数量的样本
    用于模拟工业现场新缺陷样本量不足的真实工况
    策略：每个新类随机抽取 N 张图片（优先选含该类标注的图）
    """
    def __init__(self, full_dataset, new_class_global_ids, samples_per_class,
                 seed=42):
        """
        Args:
            full_dataset: 完整的 NEUDETDataset (包含所有新类的全部样本)
            new_class_global_ids: 新类的全局类别 ID 列表 (如 [2, 4, 5])
            samples_per_class: 每个新类采样的图片数量 (如 5, 10, 30)
            seed: 随机种子（保证可复现）
        """
        self.full_dataset = full_dataset
        self.new_classes = new_class_global_ids
        self.samples_per_class = samples_per_class
        self._transforms = full_dataset._transforms
        # 构建局部→全局映射表（full_dataset 的 labels 是局部 ID，需转为全局 ID 才能比较）
        # full_dataset.global_to_local = {global_id: local_id}
        # 反转为 local_to_global = {local_id: global_id}
        self.local_to_global = {
            lid: gid for gid, lid in full_dataset.global_to_local.items()
        }
        # 按类别收集候选图片索引
        rng = np.random.RandomState(seed)
        class_to_indices = {cls_id: [] for cls_id in new_class_global_ids}
        # 读取原始标注，避免随机裁剪/增强改变候选集或消耗训练随机数。
        for idx in range(len(full_dataset)):
            if hasattr(full_dataset, 'get_global_class_ids'):
                global_labels = full_dataset.get_global_class_ids(idx)
            else:
                _, target = full_dataset[idx]
                global_labels = {
                    self.local_to_global.get(lbl, lbl)
                    for lbl in target.get('labels', torch.empty(0, dtype=torch.long)).tolist()
                }
            for global_lbl in global_labels:
                if global_lbl in class_to_indices:
                    class_to_indices[global_lbl].append(idx)
        # 每类采样指定数量的图片
        self.selected_indices = []
        for cls_id in new_class_global_ids:
            candidates = class_to_indices.get(cls_id, [])
            n = min(samples_per_class, len(candidates))
            if n > 0:
                chosen = rng.choice(candidates, size=n, replace=False).tolist()
```

```
                self.selected_indices.extend(chosen)
                print(f"[SmallSample] Class {NEU_DET_CLASSES[cls_id]}({cls_id}): "
                      f"{n}/{len(candidates)} images selected")
            else:
                print(f"[SmallSample] WARNING: No images found for "
                      f"class {NEU_DET_CLASSES[cls_id]}({cls_id})")
        # 去重（一张图可能含多个新类）
        self.selected_indices = sorted(list(set(self.selected_indices)))
        print(f"[SmallSample] Total unique images: {len(self.selected_indices)} "
              f"(target: {samples_per_class} x {len(new_class_global_ids)} = {samples_per_class
     * len(new_class_global_ids)})")
    def __len__(self):
        return len(self.selected_indices)
    def __getitem__(self, idx):
        real_idx = self.selected_indices[idx]
        return self.full_dataset[real_idx]
def build_small_sample_dataset(args, phase_classes, new_class_global_ids,
                               samples_per_class, image_set='train'):
    """构建小样本数据集的便捷函数"""
    full_dataset = build_neu_det_dataset(image_set, args, phase_classes=phase_classes)
    small_dataset = SmallSampleNEUDETDataset(
        full_dataset=full_dataset,
        new_class_global_ids=new_class_global_ids,
        samples_per_class=samples_per_class,
        seed=args.seed
    )
    return small_dataset
# 文件 tools/run_incremental_task.py
"""Execute a validated UI task independently of the Gradio process."""
import argparse
import json
import os
import sys
import time
import traceback
from pathlib import Path
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
os.environ.setdefault("PYTHONIOENCODING", "utf-8")
os.environ.setdefault("MPLBACKEND", "Agg")
os.environ["NEU_VAL_SIZE"] = "224"
from services.workflow_store import WorkflowStore, digest, timestamp, write_json
class TaskStopped(Exception):
    pass
def run(store, task_id):
    import torch
    from torch.utils.data import DataLoader
    from adapters.deformable_detr import CLASSES, evaluate, inspect_checkpoint, load_model
    from datasets.neu_det import NEUDETDataset, make_neu_det_transforms
    from incremental_train import IncrementalTrainer
```

```
    from services.dataset_service import verify_snapshot
    from services.task_service import export_package
    from util.misc import collate_fn
    task = store.task(task_id)
    task_dir = Path(task["task_dir"])
    config = task["config"]
    if digest(task_dir / "task_config.json") != task["config_sha256"]:
        raise ValueError("任务配置校验失败。")
    if digest(task_dir / "dataset_manifest.json") != task["manifest_sha256"]:
        raise ValueError("样本清单校验失败。")
    manifest = json.loads((task_dir / "dataset_manifest.json").read_text(encoding="utf-8"))
    verify_snapshot(task_dir, manifest)
    if digest(config["parent_checkpoint"]) != config["parent_sha256"]:
        raise ValueError("父模型权重校验失败。")
    if config["device"] == "cuda" and not torch.cuda.is_available():
        raise ValueError("当前环境没有可用 CUDA，请使用 CPU 或检查环境。")
    def check_stop():
        if store.task(task_id)["state"] == "stop_requested":
            raise TaskStopped()
    def emit(stage, **fields):
        store.event(task_id, stage, **fields)
        store.update_task(task_id, stage=stage, **{key: value for key, value in fields.items()
                                                   if key in ("epoch", "epochs", "batch", "batch
    es", "frozen_tensors")})
        if stage in ("本轮已保存", "分支初始化"):
            check_stop()
    check_stop()
    emit("数据准备完成", images={role: len(manifest[role]) for role in ("new", "replay", "valid
    ation")})
    # Seed before construction so newly initialized parameters remain reproducible.
    torch.manual_seed(config["seed"])
    model, args, parent_info = load_model(config["parent_checkpoint"], config["device"],
                                          ",".join(config["parent_class_names"]), config["parent
    _model_config"])
    args.incremental_arch = "dual"
    args.split_group = config["split_group"]
    args.warmstart_epochs = 0
    args.seed = config["seed"]
    args.epochs_incremental = config["epochs"]
    args.batch_size = config["batch_size"]
    args.num_workers = 0
    args.output_dir = str(task_dir / "output")
    args.new_class_samples = config["samples"]
    args.dual_lr, args.dual_head_lr = config["dual_lr"], config["dual_head_lr"]
    args.accum_steps_inc = config["accum_steps"]
    args.dual_dn_groups = 0
    args.dual_quality_loss = False
    args.cls_loss_coef, args.bbox_loss_coef, args.giou_loss_coef = 2., 5., 2.
    args.set_cost_class, args.set_cost_bbox, args.set_cost_giou = 2., 5., 2.
    args.resume = config["parent_checkpoint"]
```

```
trainer = IncrementalTrainer(args)
def dataset(role, training):
    folder = task_dir / "dataset" / role
    transforms = make_neu_det_transforms("train" if training else "val",
                                         rot_new_ids=manifest["new_class_ids"] if role == "
new" else None,
                                         val_size=224)
    return NEUDETDataset(folder / "images", folder / "labels", list(range(6)), transforms=t
ransforms)
trainer.workflow_new_dataset = dataset("new", True)
trainer.workflow_replay_dataset = dataset("replay", True)
trainer.workflow_validation_dataset = dataset("validation", False)
validation_loader = DataLoader(trainer.workflow_validation_dataset, batch_size=4, shuffle=F
alse,
                               num_workers=0, collate_fn=collate_fn)
emit("基座评估")
before = evaluate(model, validation_loader, trainer.device, manifest["old_class_ids"], mani
fest["new_class_ids"])
write_json(task_dir / "baseline_metrics.json", before)
emit("基座评估完成", metrics=before)
check_stop()
emit("回放准备完成", images=len(manifest["replay"]), policy="seeded_random_pure_old")
trainer.workflow_event = emit
last_progress = [0.]
def progress(epoch, batch, batches, loss, lr):
    if time.monotonic() - last_progress[0] < 1 and batch != batches:
        return
    last_progress[0] = time.monotonic()
    fields = {"epoch": epoch, "batch": batch, "batches": batches, "loss": loss, "lr": lr}
    if torch.cuda.is_available() and trainer.device.type == "cuda":
        fields["gpu_memory_mib"] = round(torch.cuda.max_memory_allocated() / 1024**2, 1)
    store.event(task_id, "批次进度", **fields)
    store.update_task(task_id, **fields)
trainer.workflow_progress = progress
trainer.train_incremental_phase(model, manifest["old_class_ids"], manifest["new_class_ids"]
,
                                phase_id=1, new_class_samples=config["samples"])
check_stop()
emit("最佳模型复核")
store.update_task(task_id, state="evaluating")
del model, trainer
if torch.cuda.is_available():
    torch.cuda.empty_cache()
checkpoint_path = task_dir / "output" / "best_overall_phase1.pth"
info = inspect_checkpoint(checkpoint_path)
restored, restored_args, _ = load_model(checkpoint_path, config["device"])
after = evaluate(restored, validation_loader, restored_args.device,
                 manifest["old_class_ids"], manifest["new_class_ids"])
saved = info["checkpoint_metrics"]
for key in ("old_ap", "new_ap", "overall_ap"):
```

```
        if saved.get(key) is None or after[key] is None or abs(saved[key] - after[key]) > 1e-5:
            raise ValueError(f"最佳存档复算指标不一致：{key}，保存={saved.get(key)}，重算={afte
    r.get(key)}")
    result = {"before": before, "after": after, "parent_version_id": config["parent_version_id"
    ],
              "checkpoint": str(checkpoint_path), "checkpoint_sha256": info["sha256"],
              "selected_epoch": info["epoch"] + 1, "manifest_sha256": config["manifest_sha256"]
    ,
              "old_ap_change": after["old_ap"] - before["old_ap"],
              "metric_protocol": config["metric_protocol"]}
    write_json(task_dir / "comparison.json", result)
    check_stop()
    info.update(name=task["name"] + " · V1", project_id="neu-det", parent_version_id=config["pa
    rent_version_id"],
                task_id=task_id, task_type="class_incremental", metrics=after,
                old_class_ids=manifest["old_class_ids"], new_class_ids=manifest["new_class_ids"
    ],
                manifest_sha256=config["manifest_sha256"], config=config,
                provenance={"kind": "workflow_training", "task_id": task_id,
                            "training_manifest": str(task_dir / "dataset_manifest.json")})
    version = store.register_version(info)
    package = export_package(store, version)
    emit("版本保存完成", version_id=version["id"])
    store.update_task(task_id, state="succeeded", stage="完成", finished=timestamp(),
                      result_version_id=version["id"], comparison=result, model_package=str(pac
    kage))
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--store", required=True)
    parser.add_argument("--task-id", required=True)
    args = parser.parse_args()
    store = WorkflowStore(args.store)
    try:
        run(store, args.task_id)
    except TaskStopped:
        store.event(args.task_id, "任务已停止")
        store.update_task(args.task_id, state="stopped", stage="已停止", finished=timestamp())
    except BaseException as exc:
        traceback.print_exc()
        store.event(args.task_id, "任务失败", error=str(exc))
        store.update_task(args.task_id, state="failed", stage="失败", error=str(exc), finished=
    timestamp())
        raise
if __name__ == "__main__":
    main()
# 文件 util/gradio_compat.py
"""Local compatibility for Gradio's schemas and Windows upload completion.
Pydantic can emit additionalProperties=True in file component metadata. Gradio
4.44's client assumes every child schema is a dict and crashes even on GET /.
The schema shim leaves request validation unchanged. Upload middleware delays
```

```
the upload response until Gradio's background cache move has completed.
Upstream issue: https://github.com/gradio-app/gradio/issues/10792
"""
from functools import wraps
import gradio as gr
from starlette.middleware import Middleware
class CompleteUploadsBeforeResponse:
    """Finish Gradio's background upload move before advertising its path.
    On Windows a repeated upload cannot rename over an existing cache file.
    Gradio 4.44 then moves it in a response background task. Buffer only the
    small /upload response so a following callback never opens a partial file.
    Request bodies, SSE, and other responses retain their normal streaming.
    """
    def __init__(self, app):
        self.app = app
    async def __call__(self, scope, receive, send):
        if scope["type"] != "http" or scope.get("method") != "POST" or not scope.get("path", ""
    ).endswith("/upload"):
            return await self.app(scope, receive, send)
        response = []
        async def buffer(message):
            response.append(message)
        await self.app(scope, receive, buffer)
        for message in response:
            await send(message)
class WorkflowBlocks(gr.Blocks):
    """Apply upload readiness to every server launched from this UI."""
    @wraps(gr.Blocks.launch)
    def launch(self, *args, app_kwargs=None, **kwargs):
        options = dict(app_kwargs or {})
        options["middleware"] = [Middleware(CompleteUploadsBeforeResponse), *options.get("middl
    eware", [])]
        return super().launch(*args, app_kwargs=options, **kwargs)
def support_boolean_schemas():
    from gradio_client import utils
    original = utils._json_schema_to_python_type
    try:
        original(True, {})
    except TypeError:
        pass
    else:
        return  # Already supported by the installed client (or this shim).
    @wraps(original)
    def convert(schema, defs=None):
        if isinstance(schema, bool):
            return "Any" if schema else "Never"
        return original(schema, defs)
    utils._json_schema_to_python_type = convert
# 文件 util/detection_metrics.py
"""COCO AP50 on the same postprocessed predictions used by the legacy report."""
```

```
import contextlib
import io
import numpy as np
import torch
from torchvision.ops import nms as tv_nms
from util import box_ops
# Preserve the six-class legacy protocol for existing training logs.
OLD_CLASSES = [0, 1, 3]
N_CLS = 6
TOPK = 100
IOU_TH = 0.5
def postprocess_variant(pred_logits, pred_boxes, orig_size, mode, quota=0, score_scale=None):
    """返回 (boxes_px [N,4], labels [N], scores [N])
    mode in {agnostic, perclass, none, quota, quota_pc}
    quota / quota_pc: 每个旧类额外保底 top-quota 个候选（绕开全局 top-100
    排序），quota_pc 再叠加 per-class NMS。
    score_scale: {class_idx: 倍数} per-class 分数校准，在 NMS 前应用。
    """
    prob = pred_logits.sigmoid()                    # [Q, C]
    boxes = box_ops.box_cxcywh_to_xyxy(pred_boxes)  # [Q,4] 归一化
    h, w = int(orig_size[0]), int(orig_size[1])
    scale = torch.tensor([w, h, w, h], dtype=torch.float32)
    boxes_px = boxes * scale
    flat = prob.reshape(-1)
    k = min(TOPK, flat.numel())
    topv, topi = torch.topk(flat, k)
    qi = topi // N_CLS
    li = topi % N_CLS
    b = boxes_px[qi]
    s = topv
    if mode in ("quota", "quota_pc") and quota > 0:
        e_q, e_l, e_s = [], [], []
        for c in OLD_CLASSES:
            vc, ic = torch.topk(prob[:, c], quota)
            e_q.append(ic)
            e_l.append(torch.full_like(ic, c))
            e_s.append(vc)
        qi = torch.cat([qi] + e_q)
        li = torch.cat([li] + e_l)
        s = torch.cat([s] + e_s)
        b = boxes_px[qi]
        key = qi * N_CLS + li
        order = torch.argsort(key)
        ks = key[order]
        first = torch.ones_like(ks, dtype=torch.bool)
        first[1:] = ks[1:] != ks[:-1]
        sel = order[first]
        qi, li, s, b = qi[sel], li[sel], s[sel], b[sel]
    # per-class 分数校准：在 quota 拼接之后、NMS 之前应用，
    # 保证 quota 保底捞回的低分候选同样被缩放
```

```
    if score_scale:
        fac = torch.tensor([score_scale.get(int(l), 1.0) for l in li],
                           dtype=torch.float32)
        s = s * fac
    perclass = mode in ("perclass", "quota_pc")
    if mode == "none":
        keep = torch.arange(len(s))
    elif perclass:
        keep_list = []
        for c in range(N_CLS):
            m = li == c
            if m.sum() == 0:
                continue
            sub = tv_nms(b[m], s[m], iou_threshold=0.5)
            keep_list.append(torch.nonzero(m).squeeze(1)[sub])
        keep = torch.cat(keep_list) if keep_list else torch.zeros(0, dtype=torch.long)
    else:
        keep = tv_nms(b, s, iou_threshold=0.5)
    return (b[keep].numpy(), li[keep].numpy().tolist(), s[keep].numpy().tolist())
def ap_from_preds(per_img_preds, per_img_gts, cls, oracle=False, iou_th=IOU_TH):
    """与 compute_simple_map 相同的贪心匹配 AP，oracle=True 时按 IoU 降序排（排序上限）
    iou_th: 判 TP 的 IoU 阈值，默认 0.5，可扫描低阈值看框质量上限"""
    all_scores, all_tp, n_gt = [], [], 0
    for (pb, pl, ps), gts in zip(per_img_preds, per_img_gts):
        cls_gt = gts.get(cls, [])
        n_gt += len(cls_gt)
        idx = [i for i, l in enumerate(pl) if l == cls]
        if not idx:
            continue
        if not cls_gt:
            for i in idx:
                all_scores.append(ps[i]); all_tp.append(False)
            continue
        pb_arr = np.asarray(pb, dtype=np.float64)
        ious, bestgt = [], []
        for i in idx:
            bb = pb_arr[i]
            cur = []
            for g in cls_gt:
                x1 = max(bb[0], g[0]); y1 = max(bb[1], g[1])
                x2 = min(bb[2], g[2]); y2 = min(bb[3], g[3])
                inter = max(0.0, x2 - x1) * max(0.0, y2 - y1)
                a1 = max(0.0, bb[2] - bb[0]) * max(0.0, bb[3] - bb[1])
                a2 = max(0.0, g[2] - g[0]) * max(0.0, g[3] - g[1])
                cur.append(inter / (a1 + a2 - inter + 1e-9))
            bg = int(np.argmax(cur))
            ious.append(cur[bg]); bestgt.append(bg)
        order = sorted(range(len(idx)), key=lambda j: ious[j] if oracle else ps[idx[j]],
                       reverse=True)
        matched = set()
```

```
        for oj in order:
            s = ps[idx[oj]]
            if ious[oj] > iou_th and bestgt[oj] not in matched:
                all_scores.append(s); all_tp.append(True); matched.add(bestgt[oj])
            else:
                all_scores.append(s); all_tp.append(False)
    if n_gt == 0 or not all_scores:
        return 0.0, n_gt, sum(all_tp)
    od = np.argsort(np.asarray(all_scores))[::-1]
    tp = np.asarray(all_tp)[od]
    tpc = np.cumsum(tp.astype(int)); fpc = np.cumsum((~tp).astype(int))
    prec = tpc / np.maximum(tpc + fpc, 1e-10)
    rec = tpc / n_gt
    ap = float(np.trapezoid(prec, rec)) if len(prec) > 1 else float(prec[0])
    return ap, n_gt, int(tp.sum())
def coco_ap50(predictions, targets, class_names):
    from pycocotools.coco import COCO
    from pycocotools.cocoeval import COCOeval
    from util.box_ops import box_cxcywh_to_xyxy
    import torch
    annotations, images, detections = [], [], []
    for image_id, (pred, target) in enumerate(zip(predictions, targets), 1):
        height, width = map(int, target["orig_size"].tolist())
        images.append({"id": image_id, "height": height, "width": width})
        boxes = box_cxcywh_to_xyxy(target["boxes"].cpu()) * torch.tensor([width, height, width,
     height])
        for box, label in zip(boxes.tolist(), target["labels"].tolist()):
            x1, y1, x2, y2 = box
            annotations.append({"id": len(annotations) + 1, "image_id": image_id,
                                "category_id": label, "bbox": [x1, y1, x2-x1, y2-y1],
                                "area": (x2-x1)*(y2-y1), "iscrowd": 0})
        for box, label, score in zip(*pred):
            x1, y1, x2, y2 = map(float, box)
            detections.append({"image_id": image_id, "category_id": int(label),
                               "bbox": [x1, y1, x2-x1, y2-y1], "score": float(score)})
    with contextlib.redirect_stdout(io.StringIO()):
        gt = COCO()
        gt.dataset = {"images": images, "annotations": annotations, "info": {},
                      "categories": [{"id": i, "name": name} for i, name in enumerate(class_nam
    es)]}
        gt.createIndex()
        if detections:
            dt = gt.loadRes(detections)
        else:
            dt = COCO()
            dt.dataset = {**gt.dataset, "annotations": []}
            dt.createIndex()
        evaluator = COCOeval(gt, dt, "bbox")
        evaluator.params.iouThrs = np.array([.5])
        evaluator.evaluate()
```

```
        evaluator.accumulate()
    result = {}
    for index in range(len(class_names)):
        precision = evaluator.eval["precision"][0, :, index, 0, -1]
        valid = precision[precision > -1]
        result[index] = float(valid.mean()) if len(valid) else None
    return result
# 文件 util/checkpoint_config.py
"""Rebuild incremental models with the topology and gates used in training."""
from collections.abc import Mapping
import numpy as np
import torch
MODEL_ARGUMENTS = (
    "backbone", "dilation", "num_feature_levels", "enc_layers", "dec_layers",
    "dim_feedforward", "hidden_dim", "dropout", "nheads", "num_queries",
    "dec_n_points", "enc_n_points", "with_box_refine", "two_stage", "aux_loss",
    "position_embedding", "position_embedding_scale", "masks",
)
def _saved_args(checkpoint):
    saved = checkpoint.get("args", {})
    if saved is None:
        return {}
    return dict(saved) if isinstance(saved, Mapping) else vars(saved)
def model_config(model, args):
    if hasattr(model, "_orig_mod"):
        model = model._orig_mod
    config = {key: getattr(args, key) for key in MODEL_ARGUMENTS if hasattr(args, key)}
    config.update(
        incremental_arch=("dual" if hasattr(model, "new_branch") else
                          "adapter" if model.head_adapters is not None else "baseline"),
        adapter_dim=model.adapter_dim,
        adapter_dropout=model.adapter_dropout,
        num_queries=model.num_queries,
        shape_gate_ids=model.shape_gate_ids,
        adapter_class_ids=model.adapter_class_ids,
        class_global_ids=model.class_global_ids,
    )
    if hasattr(model, "new_branch"):
        dn = model.new_branch.denoising
        config.update(dual_old_class_ids=model.old_class_ids,
                      dual_new_class_ids=model.new_class_ids,
                      new_num_queries=model.new_branch.query_embed.num_embeddings,
                      dual_dn_groups=dn.groups if dn is not None else 0,
                      dual_dn_box_noise=dn.box_noise if dn is not None else .4,
                      dual_dn_label_noise=dn.label_noise if dn is not None else .2,
                      dual_dn_weight=getattr(args, "dual_dn_weight", 1.),
                      dual_quality_loss=getattr(args, "dual_quality_loss", False))
    return config
def apply_checkpoint_model_args(args, checkpoint, keep_requested_arch=False):
    """For base training, preserve the user's requested new adapter architecture."""
```

```
    config = {**_saved_args(checkpoint), **checkpoint.get("model_config", {})}
    state = checkpoint.get("model", checkpoint)
    for key in MODEL_ARGUMENTS:
        if key in config:
            setattr(args, key, config[key])
    if "query_embed.weight" in state:
        args.num_queries = state["query_embed.weight"].shape[0]
    if not keep_requested_arch:
        # Weights determine whether adapters exist, including older raw state dicts.
        down = state.get("head_adapters.classification.down.weight")
        dual_query = state.get("new_branch.query_embed.weight")
        args.incremental_arch = "dual" if dual_query is not None else "adapter" if down is not
    None else "baseline"
        if dual_query is not None:
            if not all(key in config for key in ("dual_old_class_ids", "dual_new_class_ids")):
                raise RuntimeError("Dual checkpoint is missing its class partition metadata")
            args.dual_old_class_ids = list(config["dual_old_class_ids"])
            args.dual_new_class_ids = list(config["dual_new_class_ids"])
            args.new_num_queries = dual_query.shape[0]
            has_dn = "new_branch.denoising.label_embed.weight" in state
            if has_dn and not config.get("dual_dn_groups", 0):
                raise RuntimeError("DN checkpoint is missing its denoising configuration")
            args.dual_dn_groups = config.get("dual_dn_groups", 0) if has_dn else 0
            args.dual_dn_box_noise = config.get("dual_dn_box_noise", .4)
            args.dual_dn_label_noise = config.get("dual_dn_label_noise", .2)
            args.dual_dn_weight = config.get("dual_dn_weight", 1.)
            args.dual_quality_loss = config.get("dual_quality_loss", False)
        if down is not None:
            args.adapter_dim = down.shape[0]
            args.adapter_dropout = config.get("adapter_dropout", 0.0)
    return args
def restore_model_metadata(model, checkpoint):
    config = checkpoint.get("model_config", {})
    if "shape_gate_ids" in config:
        model.set_shape_gate_ids(config["shape_gate_ids"])
    elif checkpoint.get("phase", 0) > 0 or "new_classes" in checkpoint:
        # Older checkpoints saved the switch in args, but omitted the class IDs.
        saved = _saved_args(checkpoint)
        if saved.get("shape_gate", 1):
            from datasets.neu_det import SPLIT_GROUPS
            group = SPLIT_GROUPS[str(saved.get("split_group", "1"))]
            classes = checkpoint.get("phase_classes", list(range(model.current_num_classes)))
            new_classes = checkpoint.get("new_classes", group["new"])
            model.set_shape_gate_ids([classes.index(c) for c in new_classes if c in classes])
    model.set_adapter_class_ids(config.get("adapter_class_ids"))
    if "class_global_ids" in config:
        model.class_global_ids = list(config["class_global_ids"])
    if hasattr(model, "new_branch"):
        if model.class_global_ids != sorted(model.old_class_ids + model.new_class_ids):
            raise RuntimeError("Dual checkpoint class order does not match its branch partition
```

```
    ")
def load_inference_weights(model, checkpoint):
    """Permit only the historically optional shape branch to be absent."""
    state = checkpoint.get("model", checkpoint)
    state = {k: v for k, v in state.items() if not k.startswith("teacher_model.")}
    # Older single-stage expansion registered a second name for the same head.
    # Accept only exact aliases; divergent weights must remain an error.
    if not model.two_stage:
        for key in list(state):
            prefix = "transformer.decoder.class_embed."
            if key.startswith(prefix):
                canonical = "class_embed." + key[len(prefix):]
                if canonical not in state or not torch.equal(state[key], state[canonical]):
                    raise RuntimeError(f"Inconsistent classifier alias in checkpoint: {key}")
                del state[key]
    current = model.state_dict()
    if "model_config" not in checkpoint:
        for key, value in current.items():
            if key.startswith("shape_delta.") and key not in state:
                state[key] = value
    model.load_state_dict(state, strict=True)
    restore_model_metadata(model, checkpoint)
def build_legacy_full_model(ckpt):
    """旧版全量训练管线存档（顶层键 model_state_dict + 自带 args）：
    with_box_refine + num_feature_levels=5 + num_queries=50 + 每层单线性类头。
    按存档 args 原样重建——当前仓库的 deformable_detr.py 即当年的 box_refine 变体。"""
    from models.backbone import build_backbone
    from models.deformable_transformer import build_deforamble_transformer
    from models.deformable_detr import DeformableDETR
    la = ckpt["args"]
    la = la if isinstance(la, dict) else vars(la)
    defaults = dict(backbone="resnet50", dilation=False, position_embedding="sine",
                    position_embedding_scale=2 * np.pi, masks=False,
                    lr_backbone=0.0, frozen_weights=None,
                    hidden_dim=256, dropout=0.1, nheads=8, enc_layers=3,
                    dec_layers=3, dim_feedforward=512, n_points=4,
                    enc_n_points=6, dec_n_points=4, num_feature_levels=5,
                    num_queries=50, two_stage=False, with_box_refine=True)
    merged = {**defaults, **la}
    class _NS:
        pass
    a = _NS()
    for k, v in merged.items():
        setattr(a, k, v)
    backbone = build_backbone(a)
    transformer = build_deforamble_transformer(a)
    return DeformableDETR(backbone, transformer, num_classes=6,
                          num_queries=a.num_queries,
                          num_feature_levels=a.num_feature_levels,
                          aux_loss=False, with_box_refine=True, two_stage=False)
```